Выпускная квалификационная работа

Устинов И. А., БИСО-01-21

Содержание

ВВЕДЕНИЕ **4**

1 ТЕОРЕТИКО-МЕТОДИЧЕСКИЕ ОСНОВЫ ПРОЕКТИРОВАНИЯ СИСТЕМЫ ЗАЩИТЫ ОТ ВРЕДОНОСНОЙ СЕТЕВОЙ АКТИВНОСТИ С ИСПОЛЬЗОВАНИЕМ ПОВЕДЕНЧЕСКОГО ПРОФИЛИРОВАНИЯ **6**

1.1 Анализ актуальных векторов сетевых атак 6

1.2 Анализ решений по защите пользователей от подозрительной сетевой активности 8

1.2.1 Мобильные анализаторы трафика 10

1.2.2 Облачные сервисы фильтрации и мониторинга на уровне DNS 10

1.2.3 DNS-серверы для локальных сетей 11

1.2.4 Мобильные средства защиты от угроз 11

1.2.5 Веб-браузеры со встроенной фильтрацией 12

1.3 Ключевые методы и технологии обнаружения вредоносной активности в трафике 14

1.3.1 Глубокий анализ пакетов 14

1.3.2 Потоковый анализ 15

1.3.3 Анализ на основе сигнатур 15

1.3.4 Применение песочниц 15

1.3.5 Поведенческий анализ и машинное обучение 15

1.4 Централизованный сбор и извлечение признаков сетевого трафика 18

2 РАЗРАБОТКА АРХИТЕКТУРЫ СИСТЕМЫ **21**

2.1 Серверная архитектура 21

2.1.1 Сбор данных о сетевых подключениях 22

2.1.2 Анализ сетевого трафика 22

2.1.3 Хранение данных 24

2.1.4 Клиент-серверное взаимодействие 24

2.1.5 Поведенческое профилирование 25

2.2 Клиентская часть 27

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ МНОГОУРОВНЕВОЙ

СИСТЕМЫ ЗАЩИТЫ ПОЛЬЗОВАТЕЛЕЙ ОТ ВРЕДОНОСНОЙ СЕТЕВОЙ АКТИВНОСТИ С ИСПОЛЬЗОВАНИЕМ ПОВЕДЕНЧЕСКОГО ПРОФИЛИРОВАНИЯ **31**
3.1 Выбор инструментов и среды разработки прототипа 32
3.2 Настройка механизмов перехвата и анализа трафика 33
3.2.1 Настройка инструмента OpenVPN 33
3.2.2 Настройка анализатора сетевого трафика Zeek 38
3.3 Конфигурация модуля хранения данных 44
3.4 Создание модуля поведенческого профилирования 51
3.5 Формирование REST API 54
3.5.1 Настройка Node.js 55
3.5.2 Файловая организация подсистемы и программная реализация модуля 55

Приложения **61**

A Скрипт парсинга status-server.log **61**

B Содержимое constants.py **63**

C Содержимое collector.py **64**

D Содержимое helpers.py **66**

E Содержимое trainer.py **68**

F Содержимое checker.py **70**

G Содержимое файла index.ts **74**

H Функция checkAuth **76**

I Функция mainAuth **77**

J Содержимое файла queryDb.ts **78**

K Содержимое файла vpnConf.ts **79**

L Содержимое файла userEntryControl.ts **81**

M Содежимое файла userDependControl.ts **86**

N Содержимое файла userNotificationControl.ts **91**

O Содержимое файла userControl.ts **94**

P Содержимое файла blockListControl.ts **96**

Q Содержимое файла alert.ts **100**

R Содержимое файла countries.ts **104**

S Содержимое файла graph.ts **107**

T Содержимое файла ssl.ts **114**

ВВЕДЕНИЕ

Актуальность темы. В современном мире невозможно обойтись без выхода в Интернет: общение, развлечения, образование – все это можно найти в онлайн-пространстве. Огромное количество людей в качестве инструмента для доступа в сеть выбирают мобильный телефон. В последнее время наблюдается резкий рост использования сервисов по созданию виртуальной частной сети, однако данная технология применяется пользователями бездумно, многие обыватели не задумываются, как такой способ подключения способен навредить их информационной безопасности. Рядовой пользователь может не догадываться о том, что используемый сервис пересылает личную информацию злоумышленнику. На фоне этого имеет место увеличение объемов скомпрометированной информации и утекших данных, это подтверждает статистика компании F6 за 2025 год: 760 млн строк с данными российских пользователей попали в утечки [1].

Степень научной разработанности. На сегодняшний день вопросам обнаружения аномалий в сетевом трафике и проектирования систем обнаружения вторжений посвящено большое количество научных работ. Глубоко изучены методы сигнатурного анализа и машинного обучения для выявления подозрительных действий в сети. Вместе с тем большинство существующих решений ориентированы на корпоративный сектор и требуют значительных вычислительных ресурсов и участия квалифицированных специалистов для правильной интерпретации журналов событий.

Проблема исследования. Отсутствие эффективного и удобного для рядового пользователя решения, которое обеспечивало бы комплексную защиту устройств от вредоносной сетевой активности.

Объектом исследования являются системы мониторинга сетевых соединений и защиты пользователей от вредоносной сетевой активности.

Предметом исследования выступают механизмы выявления аномалий в сетевом трафике с использованием поведенческого профилирования.

Целью выпускной квалификационной работы является разработка многоуровневой системы защиты пользователей от вредоносной сетевой активности с использованием поведенческого профилирования.

Для достижения цели поставлены и решены следующие задачи:

1) Проанализировать актуальные векторы сетевых атак;
2) Провести анализ существующих решений в области систем по обнаружению подозрительной активности в сетевом трафике;
3) Исследовать алгоритмы машинного обучения для реализации механизма поведенческого профилирования;
4) Изучить работу анализатора сетевого трафика Zeek;
5) Спроектировать серверную и клиентскую архитектуру системы;
6) Выполнить программную реализацию спроектированной многоуровневой системы защиты;
7) Провести тестирование разработанной системы и анализ результатов ее работы.

Методы исследования. Анализ существующих решений в области систем по обнаружению подозрительной активности в сетевом трафике, комплексный анализ современных векторов сетевых атак, разработка конвейера обработки данных и пользовательского интерфейса.

Научная новизна. Предложена многоуровневая система защиты, комбинирующая метод поведенческого профилирования со статическим анализом индикаторов компрометации.

Практическая значимость. Результаты работы позволяют внедрить комплексное решение для защиты пользователей, состоящее из серверного анализатора и мобильного приложения. Система автоматически перехватывает трафик, выявляет подозрительную активность, оповещает пользователя о потенциально нежелательном трафике через мобильное приложение.

Структура работы. Пояснительная записка состоит из введения, исследовательского раздела, технологического раздела, заключения, списка использованных источников и приложений. В исследовательском разделе рассматриваются теоретические основы построения системы защиты от вредоносной сетевой активности с использованием поведенческого профилирования: анализ актуальных векторов сетевых атак и решений по защите пользователей от подозрительной сетевой активности, традиционные подходы к выявлению подозрительной активности в сетевом трафике, применение алгоритмов машинного обучения для поиска аномалий в сетевом трафике, централизованный сбор и извлечение признаков сетевого трафика, взаимодействие с пользователем. В специальном разделе выполняется разработка архитектуры многоуровневой системы защиты пользователей от вредоносной сетевой активности с использованием поведенческого профилирования. В технологическом разделе будет представлена практическая реализация системы, описание программной структуры, настройка компонентов и результаты тестирования.

Глава 1

ТЕОРЕТИКО-МЕТОДИЧЕСКИЕ ОСНОВЫ ПРОЕКТИРОВАНИЯ СИСТЕМЫ ЗАЩИТЫ ОТ ВРЕДОНОСНОЙ СЕТЕВОЙ АКТИВНОСТИ С ИСПОЛЬЗОВАНИЕМ ПОВЕДЕНЧЕСКОГО ПРОФИЛИРОВАНИЯ

1.1 Анализ актуальных векторов сетевых атак

Несмотря на совершенствование средств защиты информации, именно конечный пользователь является главной целью злоумышленников. Ввиду роста использования российскими пользователями инструментов для организации защищенного доступа [2], прослеживается увеличение числа инцидентов информационной безопасности. Анализ актуальных векторов сетевых атак демонстрирует, что злоумышленники стали больше эксплуатировать доверие и техническую неосведомленность пользователей.

По данным компании Positive Technologies, с июля 2024-го по сентябрь 2025 года на долю России пришлось от 14% до 16% всех успешных кибератак в мире [3]. На рисунке ниже представлена динамика успешных кибератак на организации и частных лиц в России (рис. 1.1).

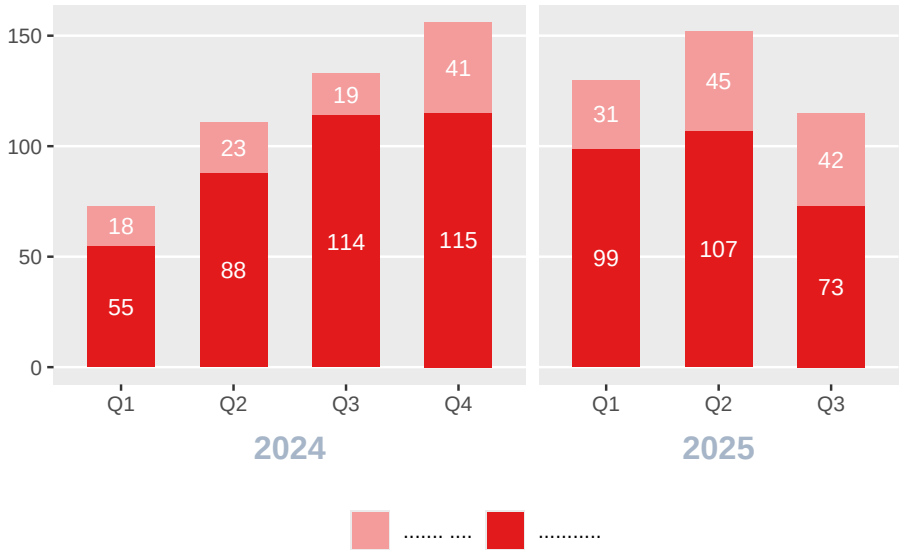


Рисунок 1.1: Динамика успешных кибератак

Исходя из данной динамики, можно сделать следующий вывод относительно частных лиц: наблюдается заметное увеличение числа атак на пользователей. Если в первой половине 2024 года фиксировалось 18-23 инцидента за квартал, то данный показатель вырос до 31-45 инцидентов в квартал за тот же временной промежуток в 2025 году, что на 53% больше.

Анализ гистограммы успешных атак относительно методов (рис. 1.2) показывает, что среди векторов воздействия преобладает социальная инженерия, на которую приходится 96%, однако стоит обратить внимание на второй по величине метод – использование вредоносного программного обеспечения. Исходя из данных наблюдений, складывается следующий вывод – злоумышленники при воздействии на частных лиц делают акцент на человеческом факторе и доверчивости пользователей.

Рассматривая распределение долей успешных атак с использованием вредоносного программного обеспечения относительно частных лиц (рис. 1.3), можно заметить преобладание следующих типов: вредоносное программное обеспечение для удаленного управления, шпионское программное обеспечение, банковский троян. Данная статистика свидетельствует о том, что злоумышленники стремятся к максимально скрытному присутствию на устройстве жертвы, незаметно для пользователя крадут его личную информацию и пересылают ее на удаленный

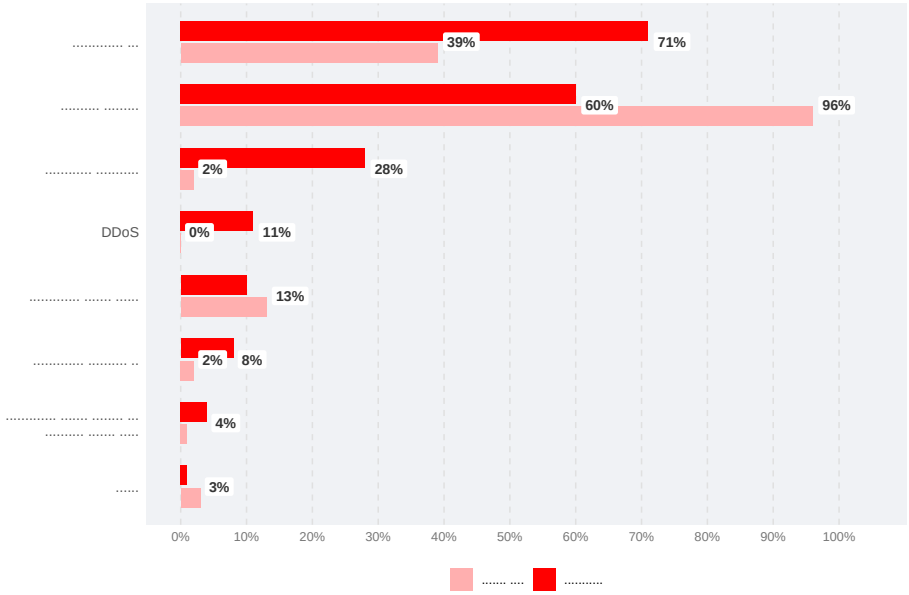


Рисунок 1.2: Методы кибератак на организации и частные лица

сервер для дальнейшей продажи.

Вывод по параграфу 1.1

Подводя итог анализу актуальных векторов кибератак, можно сделать следующий вывод относительно частных лиц: имеется тенденция к росту числа инцидентов. При атаке на пользователя злоумышленники делают акцент на человеческом факторе и технической неосведомленности граждан. Преступники стремятся к максимально незаметному присутствию на устройстве жертвы, об этом говорят преобладающие типы вредоносного программного обеспечения, а именно инструменты для удаленного управления, шпионские приложения и банковские трояны.

1.2 Анализ решений по защите пользователей от подозрительной сетевой активности

Для противодействия описанным векторам атак и защиты конечных пользователей от скрытого вредоносного воздействия применяются различные системы защиты. Архитектуру системы мониторинга сетевой активности можно представить в виде компонентов:

- Агент - установленная на отслеживаемых хостах программа, которая собирает данные о сетевой активности и отправляет их на сервер для

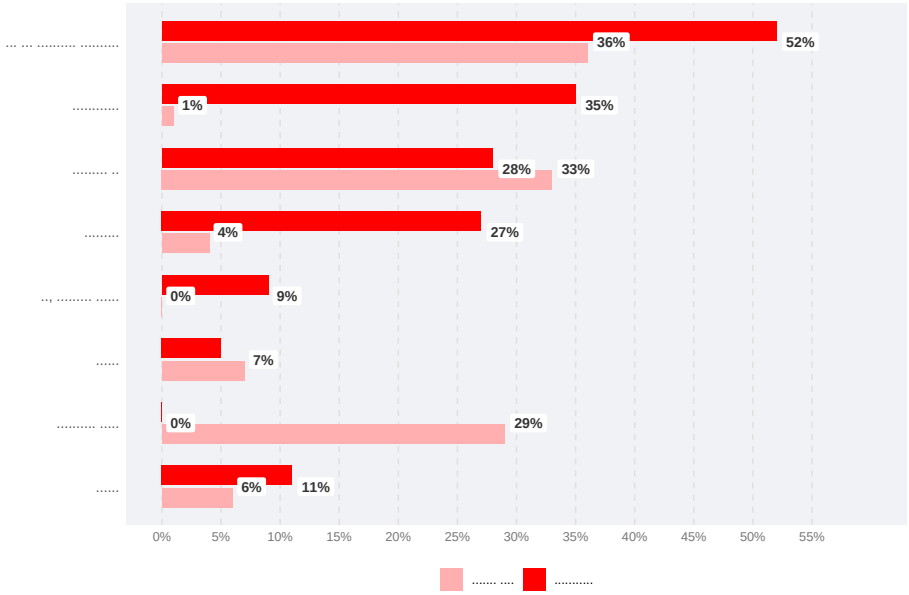


Рисунок 1.3: Типы используемого ВПО в атаках на организации и частных лиц

анализа;

- Сервер - центральный компонент системы, выполняющий анализ данных, полученных от агентов;
- База данных - централизованная хранилище, которое накапливает собранные данные о состоянии сети;
- Визуализация и управление - интерфейс, который прдоставляет пользователям доступ к информации о мониторинге и управлению настройками, чаще всего представлен в виде веб-приложения.

В процессе мониторинга сетевой активности допустимо выделить ключевые уровни контроля, зависящие от точки перехвата пакетов:

- Уровень конечной точки: мониторинг трафика и активности программ осуществялется локально на устройстве клиента;
- Уровень внутренней сети: трафик отслеживается в ммоент его прохождения через коммутатор, который связывает устройства внутри периметра;
- Уровень сетевого периметра: анализ осуществляется на границе между корпоративной инфраструктурой и Интернетом.

С учетом выделенных уровней перехвата пакетов, на сегодняшний день имеется большой выбор программных средств и архитектурных подходов, направленных на мониторинг сетевой активности пользователей.

1.2.1 Мобильные анализаторы трафика

Данные программные решения устанавливаются на смартфон клиента. Из-за соображений безопасности приложения не имеют прямого доступа к сетевому трафику телефона. Для решения проблемы подобные программные продукты используют встроенный системный механизм создания виртуальной частной сети, другими словами, мобильные приложения создают на устройстве виртуальный сетевой интерфейс и перенаправляют все сетевые пакеты через себя, то есть анализ метаданных соединений производится непосредственно в памяти устройства. Примерами такого типа приложений могут служить: GlassWire и PCAPdroid.

Главным преимуществом решения является высокая степень конфиденциальности, поскольку данные не покидают пределы устройства и пользователю не требуется доверять свой трафик сторонним серверам.

У такого подхода имеется несколько существенных недостатков. Так как выполнение анализа пакетов, сверка метаданных с индикаторами компрометации из баз данных осуществляется непосредственно на смартфоне, это создает избыточную нагрузку на процессор и оперативную память мобильного устройства, что вызывает заторможенность в работе и повышенный расход энергии аккумулятора. Также стоит отметить, что в современных устройствах имеются встроенные механизмы операционной системы, которые направлены на оптимизацию заряда батареи и могут принудительно выгружать фоновую службу анализатора из памяти.

1.2.2 Облачные сервисы фильтрации и мониторинга на уровне DNS

При использовании данного типа реализации нет необходимости в установке какого-либо стороннего программного обеспечения на устройство клиента. Пользователь прописывает в настройках мобильного телефона адрес защищенного DNS-сервера, в свою очередь, сервис сверяет все доменные имена, которые запрашивает устройство, с индикаторами компрометации из баз данных. Примерами таких сервисов могут служить: AdGuard DNS и NextDNS.

Ключевым преимуществом такого метода является простота внедрения и нелувое влияние на производительность смартфона, так как все операции по выявлению и блокировке вредоносных узлов выполняются на далненных серверах.

Данный подход имеет ряд проблем. Стоит обратить внимание на отсутствие возможности глубокого анализа содержимого сетевых пакетов, так как система не имеет доступа к полным URL-адресам, путям запросов и передаваемому контенту. Также необходимо учитывать наличие единой точки отказа – при сбоях в сети провайдера DNS пользователь может полностью потерять доступ к интернету.

1.2.3 DNS-серверы для локальных сетей

Данный подход обеспечивает защиту клиента путем развертывания сервиса фильтрации непосредственно внутри локальной сети пользователя. Для реализации необходимо установить специализированное программное обеспечение на постоянно включенное устройство, которому в локальной сети назначается статический IP-адрес.

При стандартной конфигурации маршрутизатор автоматически передает подключаемым устройствам адрес DNS-сервера интернет-провайдера или свой собственный. После внедрения сетевого фильтра описанный выше принцип работы изменяется – всем клиентам в качестве основного DNS-сервера назначается внутренний IP-адрес развернутого сетевого фильтра. В результате все устройства сети перестают обращаться к DNS-серверам провайдера напрямую и направляют трафик через общий узел, который отсекает рекламу и известные вредоносные сайты. Самым популярным программным решением, основанным на представленном подходе, является Pi-hole.

Преимуществом такого архитектурного метода выступает безагентная защита всех устройств сети одновременно. Это позволяет эффективно защищать IoT-устройства, так как на большую часть такого оборудования технически невозможно установить классическое программное обеспечение для защиты.

Рассмотренное решение обладает рядом ограничений. Слабым местом такого архитектурного метода является зависимость от одного локального сервера. Если устройство, на котором установлено фильтрующее приложение, выключится, зависнет или получит другой IP-адрес, клиенты могут потерять доступ к сайтам. Помимо этого, стоит обратить внимание на факт функционирования защиты только внутри домашнего периметра, то есть при отключении смартфона от домашней сети и переходе на мобильную передачу данных, устройство выходит из локальной сети, следовательно, перестает защищаться фильтром.

1.2.4 Мобильные средства защиты от угроз

Данный класс приложений фокусируется на комплексной защите устройства пользователя, они позволяют выявлять вредоносные программные решения, пресекать переход клиента на фишинговый ресурс и контролировать целостность системы. Примером такого программного решения может служить Kaspersky Endpoint Security. Работа подобных решений строится на комбинации трех уровней проверки:

- Анализ приложений и файлов. Средство защиты сканирует файлы в хранилище устройства, вычисляет хеш-суммы и сверяет их с базами сигнатур известных вредоносных программных обеспечений. Также могут дополнительно оценивать набор разрешений, которые запрашивает мобильное приложение;
- Оценка уязвимостей устройства. Решение проверяет конфигурацию операционной системы, например, включенный режим отладки по USB

или разрешение на установку пакетов из неизвестных источников;

- Проверка ссылок и беспроводных сетей. Для реализации защиты от фишинга и вредоносных сайтов данные программные решения используют системные службы специальных возможностей, помимо этого анализируются параметры беспроводных сетей на подозрительные настройки;

Ключевым преимуществом применения данного инструмента является тщательная проверка файлов и внутренних настроек устройства. Это помогает вовремя находить уже известные угрозы, предотвращать эксплуатацию системных уязвимостей и блокировать установку вредоносного программного обеспечения.

Использование подобных мобильных средств защиты имеет ряд ограничений. Из-за строгой изоляции приложений данные средства проверяют сами файлы на смартфоне, сверяя контрольные суммы по базам данных для поиска совпадений, но не способны полноценно анализировать сетевой трафик других программ. Возможна ситуация, при которой на устройство пользователя было установлено вредоносное приложение, зафиксированное антивирусом как легитимное. Исходя из вышеуказанной проблемы изоляции – средство защиты не сможет заметить факт передачи таким программным обеспечением личных данных на удаленный сервер злоумышленника. Также стоит обратить внимание на необходимость постоянной фоновой активности такого типа решений – из-за чего происходит нагрузка на процессор.

1.2.5 Веб-браузеры со встроенной фильтрацией

Средства защиты такого вида представляют собой приложения для просмотра веб-сайтов, в архитектуру которых интегрированы инструменты блокировки подозрительных ресурсов и рекламы. Примерами таких решений могут выступать: Brave Browser и DuckDuckGo Privacy Browser.

При открытии любого ресурса браузер предварительно сверяет URL-адрес с базами вредоносных сайтов, переводит соединение на протокол HTTPS, блокирует выполнение сторонних скриптов. Также используется механизм песочниц, который изолирует каждую вкладку от операционной системы и друг от друга, то есть, если вредоносный код выполнится на веб-странице, он останется внутри вкладки и не сможет получить доступ к файлам на устройстве или к процессам операционной системы.

Ключевым преимуществом данного решения является использование технологии песочниц, которая строго изолирует каждую открытую вкладку друг от друга и от операционной системы устройства. Благодаря такой архитектуре, даже если вредоносный код будет успешно выполнен на скомпрометированной веб-странице, результат работы останется внутри конкретной вкладки и не сможет получить несанкционированный доступ к файловой системе смартфона.

Данный подход имеет ограничения. Подобный веб-браузер контролирует и

фильтрует сетевой трафик исключительно внутри приложения и не влияет на работу остальных программ, то есть весь трафик, создаваемый этими приложениями, проходит мимо системы защиты, минуя фильтры.

В таблице ниже (табл. 1.1) представлен сравнительный анализ рассмотренных архитектурных подходов.

Таблица 1.1: Сравнительный анализ решений по защите от подозрительной сетевой активности

Подход	Примеры решений	Основные преимущества	Ключевые недостатки
Мобильные анализаторы трафика	GlassWire, PCAPdroid	Анализ соединений непосредственно на устройстве через локальный VPN-интерфейс.	Высокая нагрузка на ЦП, ОЗУ, батарею, риск принудительной выгрузки службы операционной системой.
Облачные DNS-сервисы	AdGuard DNS, NextDNS	Отсутствие необходимости устанавливать стороннее ПО на смартфон пользователя.	Невозможность глубокого анализа пакетов, наличие единой точки отказа на стороне провайдера.
Локальные DNS-серверы	Pi-hole	Централизованная фильтрация вредоносного трафика для всех устройств в пределах сети.	Защита не функционирует при выходе из защищенного периметра, зависимость от локального узла.
Мобильные средства защиты от угроз	Kaspersky Endpoint Security	Комплексный контроль: сканирование файлов, проверка уязвимостей ОС и беспроводных сетей.	Невозможность полноценного анализа трафика других программ из-за строгой изоляции приложений, постоянная фоновая нагрузка.

Подход	Примеры решений	Основные преимущества	Ключевые недостатки
Браузеры со встроенной фильтрацией	Brave, DuckDuckGo	Встроенный механизм песочниц, изоляция вкладок, принудительное использование HTTPS.	Мониторинг и фильтрация трафика ограничиваются исключительно рамками самого приложения-браузера.

Вывод по параграфу 1.2

Подводя итог анализу существующих решений по защите пользователей от подозрительной сетевой активности, можно сделать вывод о том, что ни один из рассмотренных подходов не обеспечивает полноценного и удобного мониторинга всего трафика устройства. Решения, которые устанавливаются непосредственно на смартфон, создают высокую нагрузку на процессор и увеличивают расход заряда аккумулятора телефона, помимо этого их работа ограничивается операционной системой путем выгрузки средства защиты из фона и строгой изоляцией приложений. Веб-браузеры со встроенным фильтром обрабатывают соединения исключительно внутри себя, локальные серверы перестают защищать смартфон при выходе за пределы домашней сети, а облачные DNS-сервисы не способны выполнять глубокий анализ пакетов.

1.3 Ключевые методы и технологии обнаружения вредоносной активности в трафике

Выбор архитектуры определяет только место перехвата трафика, однако не решает задачу поиска угроз. Поэтому необходимо определить наиболее эффективный метод для обнаружения вредоносной активности в полученных сетевых пакетах. Обнаружение вредоносной сетевой активности требует применения комплекса технологий, так как ни один отдельной взятый инструмент не обеспечивает абсолютной защиты. Выбор конкретного метода или их комбинации зависит от множества факторов, например, архитектуры сети, вычислительных мощностей сервера. В данном разделе рассматриваются базовые принципы работы этих методов, их ключевые преимущества и ограничения.

1.3.1 Глубокий анализ пакетов

Метод позволяет заглядывать внутрь сетевых пакетов и анализаровать их содержимое до уровня приложений, другими словами, если злоумышленник передает файлы по HTTP, глубокий анализ пакетов может извлечь эти файлы и

отправить на анализ в другую систему. Ограничением данного подхода является зашифрованный трафик, пакеты которого невозможно проанализировать.

1.3.2 Потоковый анализ

Вместо анализа каждого пакета по отдельности система агрегирует информацию о всем потоке целиком. Он показывает, какие узлы взаимодействуют между собой, как меняется распределение нагрузки и в каких сегментах сети растет активность. Анализ такого типа показывает себя эффективно для обнаружения аномалий в объеме, например, резкий рост исходящего трафика с конкретного хоста.

1.3.3 Анализ на основе сигнатур

Принцип работы подхода заключается в сопоставлении параметров и содержимого сетевых пакетов с заранее сформированной базой данных известных атак и уязвимостей. При обнаружении точного совпадения параметров трафика с индикаторами система фиксирует инцидент. Ключевым преимуществом данного метода выступает минимальный уровень ложноположительных срабатываний при обнаружении известных векторов атак. Стоит отметить существенный недостаток такого подхода – невозможность выявления ранее неизвестных угроз до момента обновления базы данных. Также можно выделить подтип сигнатурного анализа – применение черных и белых списков. Данный подход позволяет администраторам сети самостоятельно формировать и настраивать правила фильтрации с учетом специфики инфраструктуры или внутренних регламентов организации. Несмотря на простоту реализации механизма, он имеет существенный недостаток – необходимость постоянной актуализации правил при каждом изменении в сетевой топологии. Помимо этого, такой подход оказывается неэффективным, если злоумышленник скомпрометирует узел, который уже находится в белом списке.

1.3.4 Применение песочниц

Технология анализа подозрительных объектов в изолированной среде. Если система мониторинга сетевого трафика выявляет подозрительный файл, она может отпраавить этот элемент в песочницу для проверки - там файл выполняется в виртуальной среде. По поведению файла можно сделать вывод о вредоносности без использования сигнатур.

1.3.5 Поведенческий анализ и машинное обучение

Суть данного метода заключается в сборе данных о трафике в безопасный период и построении профиля нормально поведения. Затем система в реальном времени сравнивает текущий трафик с созданной конфигурацией - любое отклонение фиксируется как подозрительное. При использовании методов машинного обучения с целью выявления аномалий в сетевом трафике устраняются некоторые ограничения и проблемы анализа на основе сигнатур, например, неспособность

выявления ранее неизвестных угроз и зависимость от регулярного обновления баз данных индикаторов компрометации. Алгоритмы машинного обучения способны выявлять новые векторы атаки, адаптироваться к их постоянным изменениям и эффективно находить отклонения от нормального поведения [4].

Имеются два основных метода обучения модели:

1) Обучение с учителем – это метод, который использует размеченные данные для обучения моделей. Для работы такого типа алгоритмов используются данные с заранее заданными метками и цель процедуры – найти закономерности, которые позволят точно предсказывать эти индикаторы для новых данных [5]. Ключевым преимуществом применения такого подхода является высокая точность классификации при наличии качественно размеченных данных. Главный недостаток этого метода в контексте сетевой безопасности заключается в непрерывном видоизменении векторов атак, поэтому собрать абсолютно точную и актуальную базу данных для обучения практически невозможно.
2) В обучении без учителя модель получает данные без меток, и ее задача заключается в самостоятельном выделении группы данных. Этот метод полезен, когда задача состоит в поиске неожиданных и скрытых закономерностей в большом объеме информации [5]. Однако у такого метода имеется недостаток – текущий подход может создавать большой объем ложных срабатываний ввиду того, что любое отклонение от нормы может быть ошибочно зафиксировано как угроза.

Одним из методов обучения без учителя является алгоритм изолирующий лес. Метод строит ансамбль случайных бинарных деревьев, где данные рекурсивно делятся по случайным признакам и порогам. Подход эксплуатирует свойство аномалий: они находятся далеко от плотных областей распределения и требуют меньше операций для изоляции. Нормальные точки, расположенные в плотных областях, изолируются глубже в дереве, в свою очередь аномалии изолируются ближе к корню. На каждом шаге алгоритм случайно выбирает признак и порог между минимальным и максимальным значением этого признака в текущем подмножестве данных. Пространство делится на два региона, процесс повторяется рекурсивно для каждого региона до достижения условия остановки.

На рисунке ниже показана визуализация принципа изоляции в изолирующем лесу (рис. 1.4). Левая панель демонстрирует изоляцию аномалии одним разбиением, правая панель показывает необходимость множественных разбиений для изоляции нормальной точки внутри плотного кластера.

Ниже представлена таблица для сравнения методов, которые были упомянуты ранее (табл. 1.2).

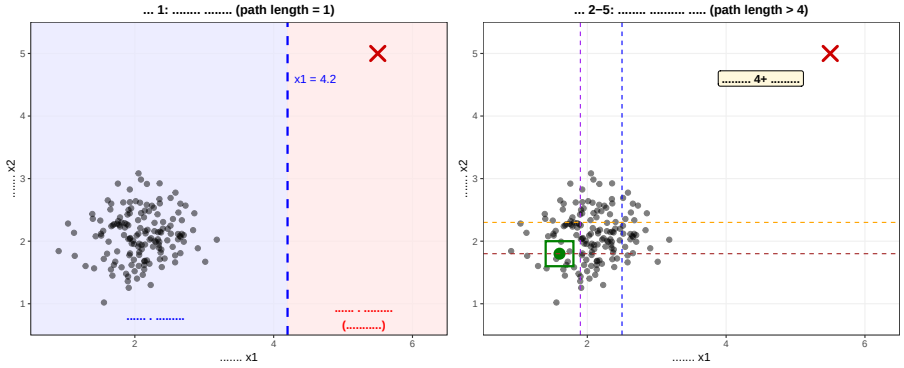


Рисунок 1.4: Принцип изоляции аномалий и нормальных точек в алгоритме Isolation Forest

Таблица 1.2: Сравнительный анализ методов обнаружения подозрительной активности

Метод	Принцип работы	Ключевые преимущества	Основные ограничения
Глубокий анализ пакетов (DPI)	Детальная инспекция содержимого пакетов до уровня приложений	Глубокая видимость передаваемых данных, высокая точность	Слепота к зашифрованному трафику (HTTPS/TLS)
Потоковый анализ	Агрегация метаданных о сетевых соединениях	Низкая нагрузка на сеть, выявление аномалий в объеме трафика	Отсутствие анализа полезной нагрузки
Сигнатурный анализ	Поиск точных совпадений с базой известных индикаторов угроз	Минимальное число ложных срабатываний, высокая скорость	Неспособность выявлять атаки нулевого дня
Песочницы	Запуск подозрительных объектов в изолированной среде	Точное определение вредоносности по реальному поведению	Задержка в анализе, высокие требования к ресурсам

Метод	Принцип работы	Ключевые преимущества	Основные ограничения
Машинное обучение	Построение профиля нормального поведения и поиск отклонений	Выявление скрытых зависимостей и ранее неизвестных угроз	Риск ложных срабатываний

Вывод по параграфу 1.3

Проведенный анализ методов обнаружения вредоносной активности в сетевом трафике показывает, что ни одна из технологий не обеспечивает абсолютное выявление аномалий. Глубокий анализ пакетов и сигнатурный метод, гарантируют высокую точность выявления известных угроз, однако уязвимы перед зашифрованным трафиком и атаками нулевого дня. Внедрение алгоритмов машинного обучения и поведенческого анализа позволяет компенсировать эти ограничения за счет выявления скрытых закономерностей и адаптации к новым векторам атак, однако остается риск ложных срабатываний.

1.4 Централизованный сбор и извлечение признаков сетевого трафика

Надежность рассмотренных методов машинного обучения напрямую зависит от качества и полносты обрабатываемой информации. Для корректной работы механизма обнаружения вторжений на основе машинного обучения необходимы данные о сетевой активности, на которых будет обучаться модель. Существуют два подхода к перехвату трафика: установка специализированного агента и использование технологии виртуальной частной сети для маршрутизации трафика на выделенный сервер.

Разработка агента является трудоемкой задачей, требующей адаптации под различные операционные системы. Использование VPN-конфигурации позволяет направить весь пользовательский трафик через выделенный сетевой интерфейс сервера, в частности применение OpenVPN гарантирует изоляцию трафика клиента от сервера, что позволяет системе анализа получить полную информацию о метаданных сетевых пакетов пользователей.

В качестве инструмента для извлечения метаданных из сетевого трафика рассматривается Zeek – программный комплекс с открытым исходным кодом для анализа трафика в сети [6]. К основным функциям и особенностям технологии можно отнести:

- Генерация большого набора логов;
- Обнаружение различных видов атак и аномалий в сети;

- Помимо функциональности «из коробки», Zeek представляет собой полностью настраиваемую и расширяемую платформу для анализа трафика. Пользователям предоставляется предметно-ориентированный язык сценариев для решения произвольных аналитических задач;

Каждый лог-файл, генерируемый инструментом, содержит в себе определенную сетевую активность: HTTP-сессии, DNS-запросы, TLS-соединение, информацию о сертификатах. На рисунке ниже представлен полный перечень возможных журналов событий, которые способна генерировать данная платформа (рис. 1.5).

В системе мониторинга сетевой активности Zeek предусмотрена возможность использования пользовательских плагинов – специализированных программных компонентов, которые расширяют базовый функционал анализатора сетевой активности. В качестве наиболее популярных плагинов можно выделить:

- Geoip-conn – данное расширение позволяет дополнить журнал событий conn.log информацией о геолокации трафика (код страны, область, город, широта, долгота)
- Zeek-intelligence-feeds – функционал данного плагина заключается в автоматическом сборе данных об индикаторах компрометации из публичных источников. Встроенный в Zeek анализатор проверяет каждый сетевой пакет на соответствие собранным данным. При обнаружении совпадения соответствующее событие фиксируется в специализированном лог-файле intel.log.

Вывод по параграфу 1.4

Таким образом, используя OpenVPN и Zeek можно сформировать конвейер данных: VPN обеспечивает прозрачный перехват пользовательского трафика, а расширяемая архитектура Zeek преобразует сырые пакеты в структурированные лог-файлы, которые можно использовать для анализа и обучения модели машинного обучения для проведения поведенческого анализа.

Выводы по первой главе

Таким образом, материалы исследовательского раздела формируют целостную теоретическую и методическую базу для дальнейшей разработки. В разделе рассмотрены актуальные векторы сетевых атак, решения по защите пользователей от подозрительной сетевой активности, традиционные подходы к выявлению аномалий, применение алгоритмов машинного обучения для поиска угроз, централизованный сбор и извлечение признаков сетевого трафика. Полученные выводы используются далее при построении специального раздела, где выполняется разработка многоуровневой системы защиты пользователей от вредоносной сетевой активности с использованием поведенческого профилирования.



Рисунок 1.5: Структура лог-файлов Zeek

Глава 2

РАЗРАБОТКА АРХИТЕКТУРЫ СИСТЕМЫ

Разрабатываемая система представляет собой программный комплекс, состоящий из серверной части и клиентской, которая реализована в виде мобильного приложения. На рисунке ниже показана ее схема (рис. 2.1).

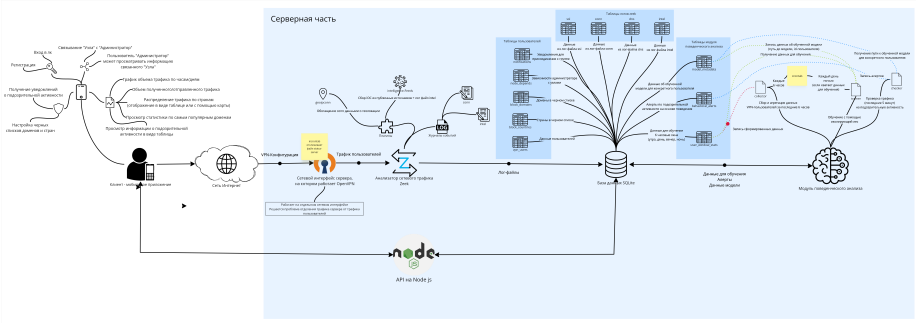


Рисунок 2.1: Схема системы

2.1 Серверная архитектура

Серверная часть выполняет полный цикл обработки данных - от захвата трафика до взаимодействия с мобильным клиентом. Логически данную область системы можно разделить на несколько компонентов:

1. Сетевой интерфейс, на котором расположен OpenVPN - необходимо для обособления сетевого трафика пользователей от серверного;

2. Анализатор Zeek - данный компонент служит инструментом для формирования журналов событий с информацией о сетевых соединениях подключенных пользователей. Также средство предоставляет плагины, которые позволяют обогащать собранную информацию сведениями об индикаторах компрометации и геолокации Ip-адресов;
3. База данных SQLite - средство для хранения информации;
4. REST API - реализует взаимодействие между клиентом и сервером;
5. Модуль поведенческого профилирования - компонент, реализующий поведенческий анализ;

2.1.1 Сбор данных о сетевых подключениях

Ранее было упомянуто про техники сбора информации о сетевых подключениях: использование виртуальной частной сети и создание специализированного агента, который будет передавать данные о сетевой активности на удаленный сервер. При реализации данной архитектуры будет использоваться второй метод, это обусловлено сложностью разработки агента для различных операционных систем. В качестве решения для создания частной виртуальной сети выбран инструмент OpenVPN. Данный выбор обоснован принципом работы программного решения, оно пропускает весь трафик через отдельный сетевой интерфейс, благодаря чему система анализа получает возможность просматривать весь трафик пользователей. Таким образом, Zeek отслеживает и записывает информацию о метаданных соединений, принадлежащих клиентам.

2.1.2 Анализ сетевого трафика

Для сбора метаданных сетевого трафика используется инструмент Zeek. Архитектура средства разделена на два основных компонента. Первый компонент – это движок событий, который помогает разделить сетевой пакет на несколько высокоуровневых событий. Они отражают сетевую активность, т.е. они описывают то, что было замечено в сетевом пакете. Компонент включает в себя ряд подкомпонентов:

- Источники ввода;
- Анализ пакетов обрабатывает протоколы более низкого уровня, начиная с канального уровня;
- Анализ сессий проверяет протоколы прикладного уровня (HTTP, FTP и т.д.);
- Анализ файлов исследует содержимое файлов, передаваемых во время сессии

Вторым основным компонентом Zeek является интерпретатор сценариев, который выполняет набор обработчиков событий, написанных на скриптовом языке Zeek. Далее описаны его возможности в контексте разрабатываемой системы.

2.1.2.1 Журналы и отчеты

Одним из главных преимуществ данной технологии является генерация большого набора журналов событий. Из огромного числа лог-файлов можно выделить следующие:

- conn.log - является одним из наиболее важных источников информации о сетевом трафике, создаваемых Zeek. В нем фиксируется каждое соединение, которое проходит через систему мониторинга. Журнал предоставляет базовую информация о сетевых потоках на транспортном и сетевом уровнях. Каждая запись содержит следующие ключевые метрики: универсальный идентификатор сессии, который позволяет связать запись в conn.log с информацией из других журналов; ip-адреса и порты участников текущего взаимодействия; длительность соединения; количество байт, которые были переданы или получены; состояние соединения;
- dns.log - журнал, который фиксирует все DNS-запросы и ответы;
- ssl.log – журнал, который фиксирует метаданные зашифрованного https-соединения, содержит следующие ключевые метрики: уникальный идентификатор сессии, для связи с другими журналами событий, доменное имя сервера, к которому клиент подключается;

2.1.2.2 Расширение функционала

В системе мониторинга сетевой активности Zeek предусмотрена возможность использования пользовательских плагинов, позволяющие расширять базовый функционал анализатора. Для определения геолокации IP-адресов возможно применение API сторонних сервисов, например, VirusTotal, однако большинство подобных платформ предоставляют доступ на платной основе. Применяя расширение geoip-conn можно дополнить журнал событий conn.log следующей информацией:

- Код страны
- Регион
- Город
- Широта
- Долгота

Указанный плагин использует базу данных GeoLite2, которая выполняет сопоставление IP-адресов с их географическим положением. Стоит обратить внимание, что система мониторинга zeek является мощным анализатором сетевого трафика, а не системой обнаружения вторжений. Основная цель данного инструмента заключается в извлечении и структурировании метаданных о сетевой активности. В стандартной конфигурации анализатор не содержит встроенный список индикаторов компрометации и фиксирует лишь факт установки соединения. Существует возможность расширения функционала путем применения специального дополнения, обеспечивающего взаимодейтсвие Zeek с внешними источниками, которые содержат базы данных актуальных индикаторов компрометации - Zeek-Intelligence-Feeds. Функционал данного

расширения заключается в автоматическом сборе данных из публичных источников, далее встроенный в zeek анализатор проверяет каждый сетевой пакет на соответствие собранным индикаторам. При обнаружении совпадения соответствующее событие фиксируется в специальном лог-файле intel.log. Плагин обрабатывает следующие типы индикаторов компрометации:

- Ip-адреса;
- Доменные имена;
- URL-адреса;
- Хеши и имена файлов;
- Адреса электронных почт.

Таким образом, применение данного расширения позволяет выявлять по сетевым индикаторам следующие классы угроз:

- Вредоносное программное обеспечение;
- Фишинговые атаки;
- Сетевые атаки;
- Утечки данных;
- Ботнет-активность;
- Потенциально нежелательные программы.

Помимо этого Zeek способен “из коробки” работать с базой данных SQLite, то есть инструмент автоматически вносит информацию о соединениях в базу данных. Под каждый лог-файл создается отдельная таблица.

2.1.3 Хранение данных

Лог-файлы, генерируемые Zeek, содержат большой объем информации, однако их анализ в исходном текстовом формате не всегда эффективен. Для упорядочивания и систематизации полученных данных целесообразно использование реляционной базы данных. Как было упомянуто ранее анализатор имеет функционал работы с базой данных SQLite, поэтому целесообразно при реализации использовать именно эту технологию для хранения информации.

2.1.4 Клиент-серверное взаимодействие

Для обеспечения взаимодействия пользователя с накопленными в базе данных сведениями требуется реализация серверного интерфейса, предоставляющего доступ к этим данным по сети. В качестве такого интерфейса выбран REST API – архитектурный стиль взаимодействия между клиентом и сервером, используя протокол HTTP. Он позволяет приложениям обмениваться данными, используя стандартные методы. В качестве платформы для реализации REST API можно использовать Node.js – инструмент с открытым исходным кодом для работы с языком JavaScript. Для взаимодействия приложения с API необходимо реализовать endpoint-ы – конечная точка сервиса, к которой клиентское приложение обращается для выполнения определенных операций или получения данных.

2.1.5 Поведенческое профилирование

Модуль поведенческого профилирования можно разделить на 3 взаимосвязанных компонента:

- Агрегация исторических данных;
- Обучение математической модели;
- Мониторинг трафика в режиме реального времени.

Все необходимые данные для обучения модели и работы модулей располагаются в соответствующих таблицах в базе данных SQLite.

2.1.5.1 Модуль сбора статистики

Для формирования базовой линии нормального поведения пользователя применяется модуль сбора статики, который необходим для излечения сырых записей о сетевых соединениях из соответствующих таблиц базы данных и агрегирования их в фиксированные исторические окна размером в 6 часов. В процессе сборе вычисляются следующие абсолютные метрики:

- Общее количество соединений - резкий рост количества сетевых соединений может сигнализировать о применении DDoS-атаки или попытке брутфорса;
- Объем переданного и полученного трафика в байтах - рост исходящего трафика может сигнализировать о краже данных, то есть пересылка информации на удаленный сервер, в свою очередь увеличение входящего трафика может указывать на загрузку вредоносной нагрузки;
- Количество уникальных IP-адресов и портов назначения - большое число обращений к уникальным ip-адресам указывает на попытку поиска других узлов в сети;
- Средняя длительность сессии - аномально длительные и непрерывные сессии характерны для поддержания каналов связи с командным сервером злоумышленника, с другой стороны короткие соединения характерны для сетевого сканирования;
- Общее количество DNS-запросов - резкий рост количества образений к DNS-серверу является индикатором DNS-туннелирования;
- Количество уникальных доменов - нетипично большое число запросов к разнообразным новым сетевым адресам часто указывает на активность вредоносного программного обеспечения. Зараженные устройства могут использовать специальные алгоритмы, которые ежедневно создают тысячи случайных адресов для поиска сервера злоумышленника и связи с ним. Из-за этого в сетевом трафике появляется множество уникальных имен, которые ранее не встречались;
- Количество ошибок NXDOMAIN - данный показатель напрямую связан с предыыдущим. Так как вредоносные программы генерируют адреса случайным образом, злоумышленники физически не могут зарегистрировать их все, следовательно, на все несуществующие dns приходит соответствующая ошибка.

Извлеченные за данный временной интервал данные сохраняются в отдельную таблицу базы данных с привязкой к конкретному дню недели и временному слоту. Перед передачей данных в алгоритм машинного обучения они проходят обязательный этап математической обработки. Система обучается на больших исторических интервалах длинной по 6 часов, однако при этом мониторинг текущего трафика в реальном времени происходит короткими отрезками, например, каждые 5 минут, следовательно, алгоритм может ошибочно классифицировать 5-минутный отрезок трафика как аномалию из-за того, что итоговые цифры оказались намного меньше тех, что ему были предоставлены при обучении. Для решения образовавшейся проблемы система переводит абсолютные значения в формат интенсивности - общее количество делится на длину временного окна в секундах. В итоге такой подход позволяет модели машинного обучения адекватно оценивать плотность и характер сетевой активности пользователя вне зависимости от того, за какой временной промежуток она была собрана.

Особое внимание в процессе подготовки данных уделяется обработке временного контекста. Традиционное представление времени в виде порядковых номеров создает проблему для алгоритмов машинного обучения, например, дню недели “понедельник” соответствует числовое значение 1, а “воскресенье” - 7, для обучаемой модели данные значения отдалены друг от друга, однако в реальности эти дни недели следуют непрерывно друг за другом. Для устранения данного разрыва применяется метод тригонометрического кодирования циклических признаков - метод, используемый в машинном обучении для представления циклических данных, таких как временные ряды или периодические признаки [7]. Для кодирования циклических данных можно использовать следующие формулы (2.1 - 2.2):

$$x_{sin} = \sin\left(2\pi * \frac{x}{P}\right) \tag{2.1}$$

$$x_{cos} = \cos\left(2\pi * \frac{x}{P}\right) \tag{2.2}$$

где:

- x: значение циклического параметра (например, понедельник = 1);
- P: период цикла (например, 7 - количество дней в неделе);
- xsin и xcos: закодированные значения, которые передаются в модель

2.1.5.2 Модуль обучения

Обучение модели запускается для каждого активного пользователя при условии наличия достаточного объема исторической выборки. В качестве основного алгоритма выявления аномалий применяется изолирующий лес, объяснение данного выбора представлено ранее. Обученная модель сохраняется в файловую систему сервера, путь к файлу фиксируется в базе данных.

2.1.5.3 Модуль мониторинга

Детектирование аномалий в режиме реального времени представляет собой непрерывный процесс оценки сетевой активности пользователей, направленный на выявление отклонений от сформированной базовой линии нормального поведения. Мониторинг осуществляется путем периодического сканирования трафика активных пользователей виртуальной частной сети каждые N минут. Если алгоритм классифицирует текущий вектор как подозрительный, сетевая активность признается аномальной, система создает соответствующую запись в таблице базы данных.

Вывод по параграфу 2.1

В ходе проектирования серверной архитектуры системы мониторинга сетевой активности сформирован концепт программного комплекса, серверная часть которого выполняет полный цикл обработки данных – от захвата трафика до взаимодействия с мобильным клиентом. Анализатор zeek сканирует сетевой интерфейс, обслуживающий трафик пользователей, подключенных через виртуальную частную сеть, для каждого его журнала событий создается отдельная таблица в базе данных SQLite. Также сервер обогащает собранные записи сведениями об индикаторах компрометации и геолокации IP-адресов. Доступ клиента к обработанных данным осуществляется через REST API. Подсистема поведенческого профилирования использует алгоритм машинного обучения “изолирующий лес” для непрерывного анализа трафика и детектирования аномалий в режиме реального времени.

2.2 Клиентская часть

В качестве инструмента взаимодействия пользователя с системой мониторинга стоит рассматривать мобильное приложение, так как оно обеспечивает непрерывный доступ к информации, то есть в случае обнаружения подозрительной сетевой активности пользователь может быть мгновенно проинформирован, например, с помощью push-уведомлений. Кроме того, современные мобильные операционные системы имеют встроенную поддержку управления VPN-конфигурациями, что значительно упрощает процесс подключения устройства к выделенному серверу.

Также по состоянию на июль 2026 года доля мобильного трафика составляет подавляющее большинство – 65,39%, тогда как на настольные компьютеры приходится лишь 33,21% (рис. 2.2) [8].

Ретроспективный анализ за годовой период подтверждает стабильность распределения доли трафика по платформам (рис. 2.3) [8].

Для обеспечения качественного взаимодействия с пользователем архитектура клиентского приложения должна базироваться на следующих принципах:

- Использование визуализации и агрегации данных. Анализатор сетевого

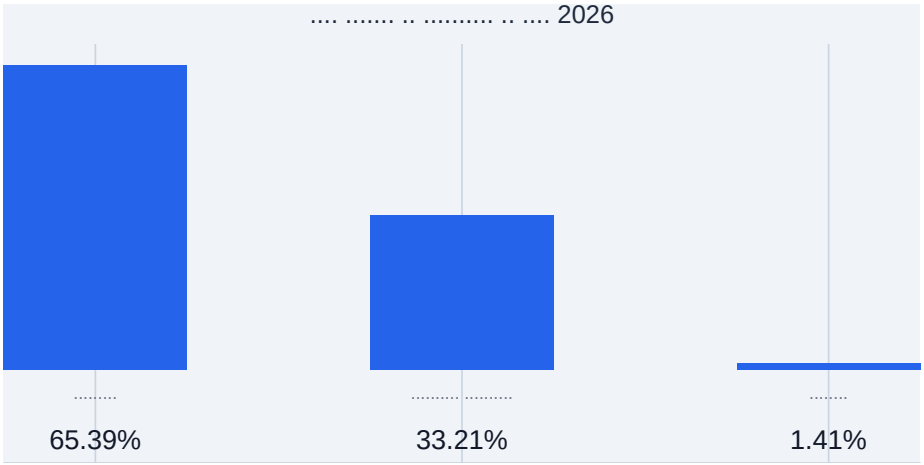


Рисунок 2.2: Доля трафика по платформам за июль 2026

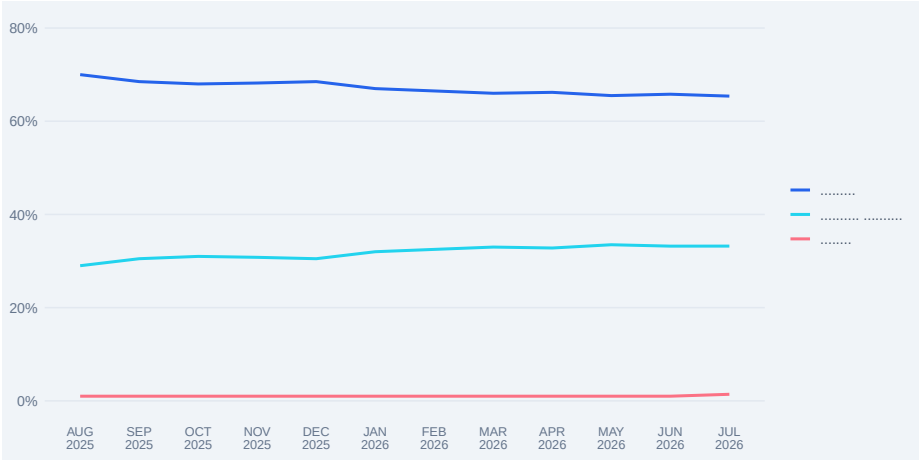


Рисунок 2.3: Динамика распределения трафика по платформам

трафика генерирует журналы событий, которые тяжело просматривать, поэтому эти данные необходимо преобразовывать в интуитивно понятные графические элементы, например, в мобильном интерфейсе можно предоставлять пользователю агрегированную статистику: графики объемов входящего и исходящего трафика, визуальное представление стран взаимодействия на карте мира или списки наиболее посещаемых доменов.

- Управление ограничениями. Система должна предоставлять пользователю возможность настраивать правила безопасности, без необходимости доступа к персональному компьютеру.

При разработке клиентской части целесообразно использовать кроссплатформенные технологии, позволяющие реализовать пользовательский интерфейс сразу для нескольких операционных систем на базе единого исходного кода, для этого можно использовать технологию React Native. Данное мобильное приложение должно включать в себя:

- Регистрацию нового пользователя;
- Вход в личный кабинет;
- Предоставление информации о сетевой активности клиента: визуализация в виде графика и расчет суммарного объема входящего и исходящего трафика за заданный временной интервал; отображение перечня стран и доменов, с которыми осуществлялось взаимодействие;
- Просмотр сведений об учетной записи пользователя с возможностью загрузки VPN-конфигурации на устройство;
- Формирование таблицы, содержащей сведения о выявленных угрозах в сетевом трафике за определенный период времени;
- Возможность создания и редактирования пользователей черных списков стран и доменов.

Вывод по параграфу 2.2

Таким образом, реализация клиентской части в виде мобильного приложения позволяет существенно повысить удобство взаимодействия пользователя с системой за счет возможности оперативного управления черными списками и просмотра информации о сетевой активности. В ходе проектирования клиентской части системы был обоснован выбор разработки кроссплатформенного мобильного приложения с использованием технологии React Native. Предложен перечень функционала, который необходимо реализовать в данном блоке программного комплекса.

Вывод по второй главе

В ходе проектирования архитектуры многоуровневой системы защиты были выполнены следующие работы:

1. Спроектирована серверная архитектура, включающая OpenVPN для изолированного перехвата пользовательского трафика, базу данных SQLite

для структурированного хранения журналов событий и REST API для обмена данными с клиентской частью;

2. Определен конвейер анализа сетевого трафика с использование инструмента Zeek и его расширений;

3. Разработана подсистема поведенческого профилирования, работающая на основе алгоритма машинного обучения без учителя;

4. Сформулированы требования к клиентской части и спроектирован функционал кроссплатформенного мобильного приложения;

Спроектированная архитектура многоуровневой системы защиты позволяет создать комплексное решение, объединяющее методы статического анализа и машинного обучения для эффективного мониторинга трафика и защиты пользователей от вредоносной сетевой активности.

Глава 3

ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ МНОГОУРОВНЕВОЙ СИСТЕМЫ ЗАЩИТЫ ПОЛЬЗОВАТЕЛЕЙ ОТ ВРЕДОНОСНОЙ СЕТЕВОЙ АКТИВНОСТИ С ИСПОЛЬЗОВАНИЕМ ПОВЕДЕНЧЕСКОГО ПРОФИЛИРОВАНИЯ

В технологическом разделе выполняется практическая реализация прототипа многоуровневой системы защиты пользователей от вредоносной сетевой активности с использованием поведенческого профилирования. Основная задача данного этапа заключается в проверке применимости архитектурных,

алгоритмических и концептуальных решений, спроектированных во второй главе. В ходе практической реализации будет описан процесс развертывания серверной инфраструктуры, включающей инструменты изоляции и анализа сетевого трафика, создание интерфейса для клиент-серверного взаимодействия, разработку модуля поведенческого профилирования на основе алгоритмов машинного обучения, а также создание мобильного приложения для пользователей. Завершающим этапом станет комплексное тестирование разработанного прототипа системы для подтверждения корректности ее работы в режиме реального времени.

3.1 Выбор инструментов и среды разработки прототипа

Для развертывания серверной части программного комплекса был выбран виртуальный выделенный сервер под управлением операционной системы Ubuntu 24.04 LTS. Вычислительные мощности включают 2 ядра процессора, 2048 МБ оперативной памяти и накопитель объемом 40960 МБ. Выбор операционной системы обусловлен ее стабильностью, долгосрочной поддержкой и совместимостью с компонентами разрабатываемой системы. Выделенных аппаратных ресурсов достаточно для обеспечения стабильной маршрутизации трафика, непрерывного анализа сетевого трафика, а также корректной работы клиент-серверного интерфейса и алгоритмов машинного обучения на этапе прототипирования и тестирования системы.

Для реализации интерфейса взаимодействия клиента с сервером была выбрана платформа Node.js, которая отличается высокой производительностью при работе с операциями ввода-вывода, также имеется возможность применения языка программирования JavaScript и его дополнения TypeScript, что позволяет создавать серверную и клиентскую части в едином стиле. В качестве фреймворка для проектирования выбран Express.js, так как поддерживает подключение к базе данных SQLite, имеет удобный и понятный синтаксис для разработки маршрутизации. В качестве основного языка разработки выбран TypeScript, так как он расширяет возможности JavaScript, добавляя строгую типизацию, следовательно, его использование снижает вероятность возникновения ошибок на этапе написания программного кода.

Модуль поведенческого профилирования реализован на языке Python. Данный язык обладает развитой экосистемой библиотек для работы с моделями машинного обучения. В процессе программной реализации модуля применяются следующие библиотеки:

- sklearn - используется для реализации алгоритма классификации “изолирующий лес”, а также для предварительной нормализации и масштабирования векторов признаков перед их подачей в модель;
- pandas - применяется для эффективной обработки данных, извлеченных из

базы данных;

- numpy - необходима для выполнения высокопроизводительных математических вычислений;
- time и datetime - для работы с временными метками;
- sqlite3 - для подключения к базе данных и выполнения SQL-запросов;
- os - для создания директории сохранения моделей машинного обучения;
- json - для упаковки подробной информации об инциденте в формат JSON;

В качестве основной среды разработки модуля поведенческого профилирования использована среда разработки PyCharm, позволяющая эффективно управлять структурой проекта, запускать программный код, отслеживать ошибки и выполнять тестирование.

В качестве системы хранения информации выбрана реляционная база данных SQLite. Использование инструмента обусловлено применением анализатора сетевого трафика Zeek и его интеграцией с данным решением. Это позволяет средству анализа автоматически экспортировать формируемые журналы событий напрямую в таблицы базы данных без необходимости разработки дополнительных механизмов интеграции.

Для реализации клиентской части разрабатываемой системы выбрана платформа React Native и язык программирования JavaScript с дополнением TypeScript. Выбор данного механизма создания клиентского интерфейса обусловлен необходимостью разработки мобильного приложения.

Учитывая многокомпонентность разрабатываемой системы, для создания клиент-серверного интерфейса и мобильного приложения была выбрана среда разработки Visual Studio Code. Такое решение обусловлено широкой экосистемой расширений данной платформы.

3.2 Настройка механизмов перехвата и анализа трафика

В данном разделе рассматривается практическая реализация подсистемы захвата сетевых пакетов, которая включает в себя маршрутизацию пользовательских данных через защищенный канал и настройку инструментов глубокого анализа трафика.

3.2.1 Настройка инструмента OpenVPN

Перехват пользовательского трафика выполняется с помощью средства Open-VPN. Принцип работы данного решения заключается в создании вртуальной частной сети, которая маршрутизирует весь сетевой трафик пользователей через выделенный сетевой интерфейс сервера. Благодаря такому подходу

система мониторинга получает доступ для просмотра сетевых пакетов, которые принадлежать исключительно клиентам.

Для настройки данного средства потребуется:

1. Закрытый ключ и сертификат для центра сертификации;
2. Серверный закрытый ключ OpenVPN, сертификат открытого ключа, подписанный в центре сертификации;
3. Клиентский закрытыый ключ, сертификат открытоого ключа, который также подписан в центре сертификации;
4. Дополнительный файл, сформированный в соответствии с протоколом Диффи-Хелламана, который используется для затруднения расшифровки перехваченного трафика при компрометации закрытого ключа злоумышленником;
5. Специальныый ключ HMAC как дополнение в целях безопасности, защищающий от различных атак на систему.

На начальном этапе конфигурации необходимо установить из официальных репозиториев Ubuntu основные пакеты: openvpn и утилиты easy-rsa, которая предназначена для создания и управления собственным центром сертификации и выпуска криптографических ключей и сертификатов.

Для обеспечения защищиенного соединения создается локальная инфраструктура удостоверяющего центра. С помощью утилиты easy-rsa выполняются следующие криптографические операции:

- Инициализация директории с открытыми ключами и генерация корневого сертификата удостоверяющего центра;
- Создание закрытого ключа и сертификата для самого сервера OpenVPN;
- Генерация параметров Диффи-Хеллмана для обеспечения безопасного обмена ключами при установке соединения;
- Генерация дополнительного ключа симметричного шифрования ta.key для защиты от DoS-атак и сканирования портов;
- Выпуск клиентских сертификатов, необходимых для последующего подключения пользователей.

В директории /etc/openvpn/ был сформирован конфигурационный файл server.conf. В нем заданы ключевые параметры работы виртуальной сети:

- Сервер просматривает входящие соединения на определенном IP-адресе. Выбран протокол транспортного уровня TCP и порт 993. Использован виртуальный сетевой интерфейс tun, который работает на сетевом уровне модели OSI;
- Указаны пути к корневому сертификату, сертификату и ключу сервера, а также параметрам Диффи-Хеллмана. Для аутентификации используется алгоритм SHA512, так же настроен параметр для проверки отозванных сертификатов;

- Клиентам выделяется подсеть 10.8.0.0/24;
- После инициализации служба понижает свои привилегии, работая от имени непривилегированного пользователя nobody и группы nogroup.

На рисунке ниже показано содержимое созданного файла server.conf (рис. 3.1).

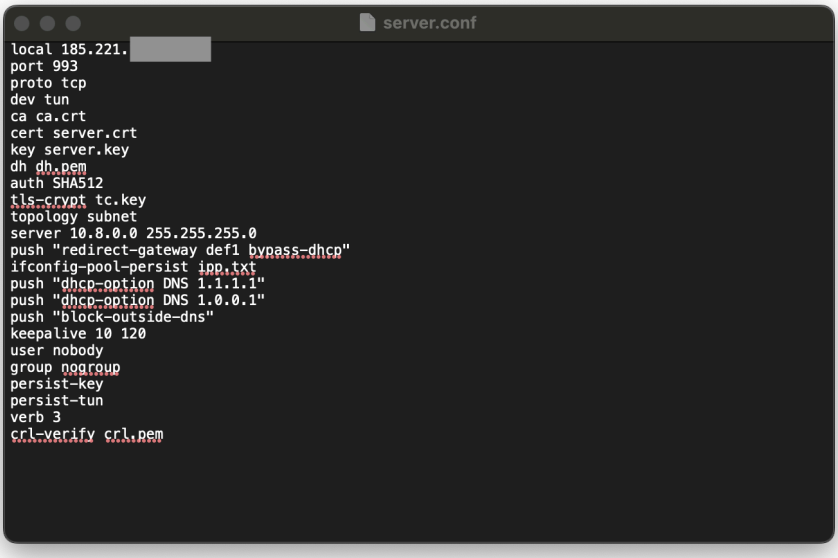


Рисунок 3.1: Содержимое файла server.conf

После генерации всех необходимых файлов директория openvpn имеет следующую структуру и наполнение (рис. 3.2).

Исходя из наполнения директории можно выделить перечень важных файлов и поддиректорий:

- server.conf - основной конфигурационный файл, определяет как сервер принимает подключения, по каким правилам шифрует трафик и какую сеть выделяет клиентам. Содержимое показано ранее (см. рис. 3.1);
- ca.crt - корневой сертификат удостоверяющего центра;
- server.crt - сертификат сервера, содержит его публичный ключ и подтверждение подлинности от удостоверяющего центра;
- server.key - закрытый ключ сервера;
- dh.pem - содержит параметры алгоритма Диффи-Хеллмана;
- ta.key и tc.key - предназначены для дополнительного шифрования;
- crl.pem - список отозванных сертификатов;
- ipp.txt - Файл пула IP-адресов. Сервер сохраняет в нем связку между идентификаторами клиентов и выданными ip-адресами, чтобы при

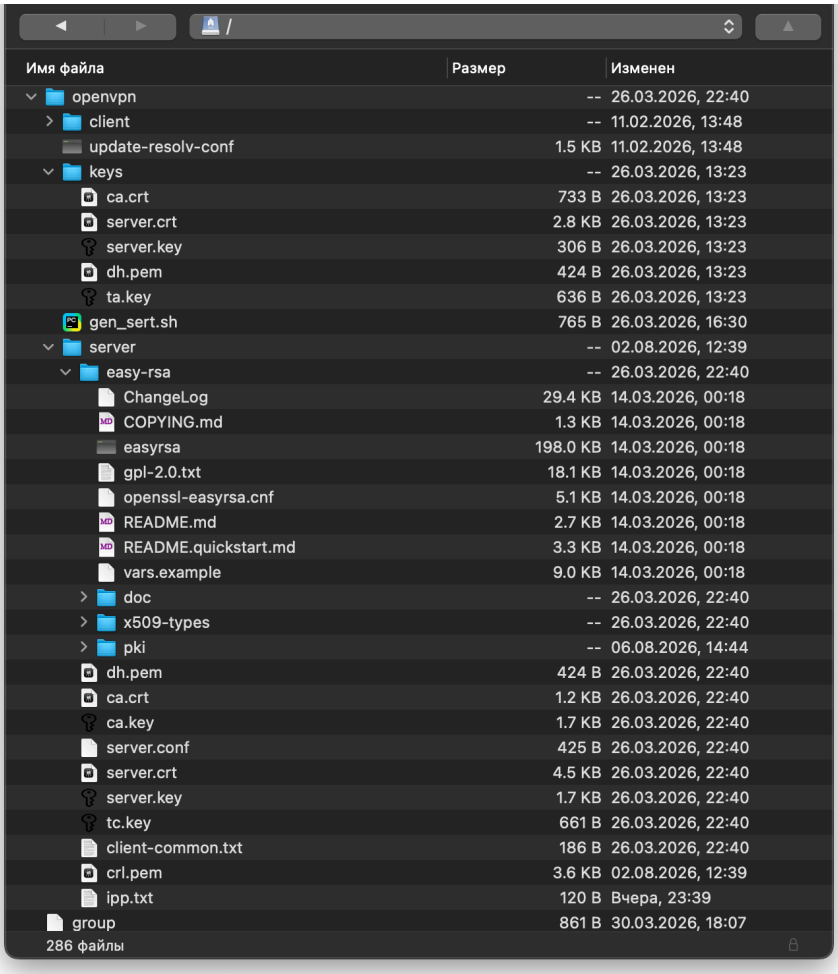


Рисунок 3.2: Наполнение директории openvpn

переподключении выдавать пользователю тот же IP;

После настройки инструмента производится проверка статуса работы (рис. 3.3)

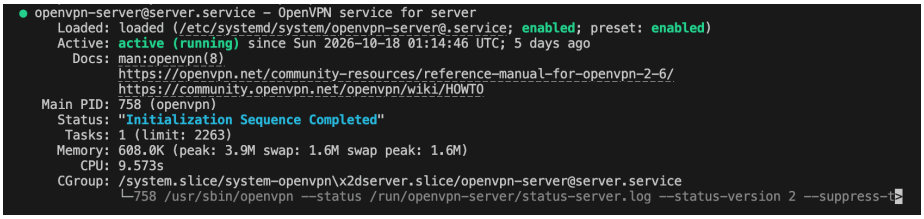


Рисунок 3.3: Статус работы сервиса

Так как подключаться к виртуальной частной сети могут несколько пользователей, то необходимо разделить их трафик для корректного анализа, с этой целью применяется инструмент incrontab и лог-файл status-server.log.

- incrontab - демон, запускающий программы по событиям файловой системы;
- status-server.log - журнал событий, содержащий статистику подключенных клиентов, которая обновляется раз в минуту, данный лог-файл содержит информацию о идентификаторе клиента, его ip-адрес (рис. 3.4).

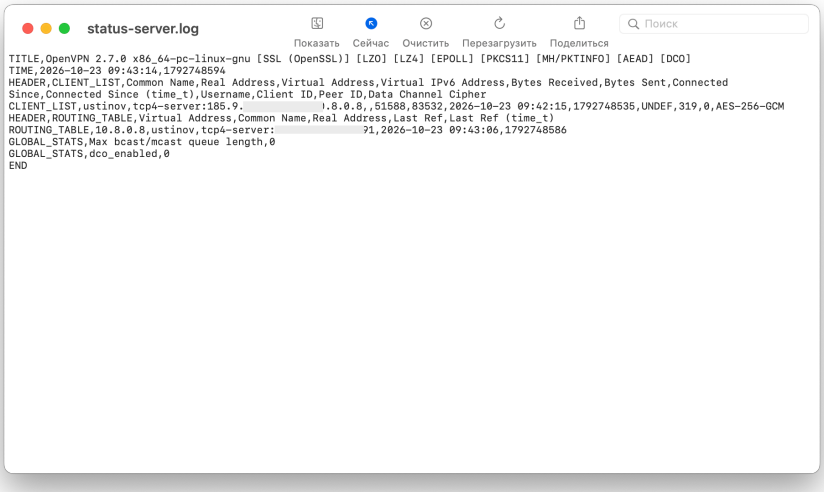


Рисунок 3.4: Содержимое status-server.log

На языке программирования Python был разработан скрипт, выполняющий парсинг status-server.log и запись сведений о VPN-пользователях в базу данных SQLite. Выполнение данного скрипта инициируется при изменении файла

status-server.log, данное событие отслеживается инструментом incrontab. Логика работы скрипта следующая:

- Если в базе данных для пользователя с соответствующим идентификатором отсутствует информация об его IP-адресе, производится его добавление;
- Если в базе данных для пользователя с соответствующим идентификатором IP-адрес присутствует и совпадает с тем, что расположен в status-server.log, то обновление или добавление новой информации в соответствующую таблицу базы даных не выполняется;

Код скрипта представлен в Приложение А

3.2.2 Настройка анализатора сетевого трафика Zeek

Для генерации подробных журналов событий о сетевых соединениях используется инструмент Zeek. По умолчанию пакет инструмента не включен в репозиторий Ubuntu по умолчанию, поэтому необходимо добавить репозиторий в APT. После установки и добавление Zeek в системную переменную производится проверка корректности установки (рис. 3.5).

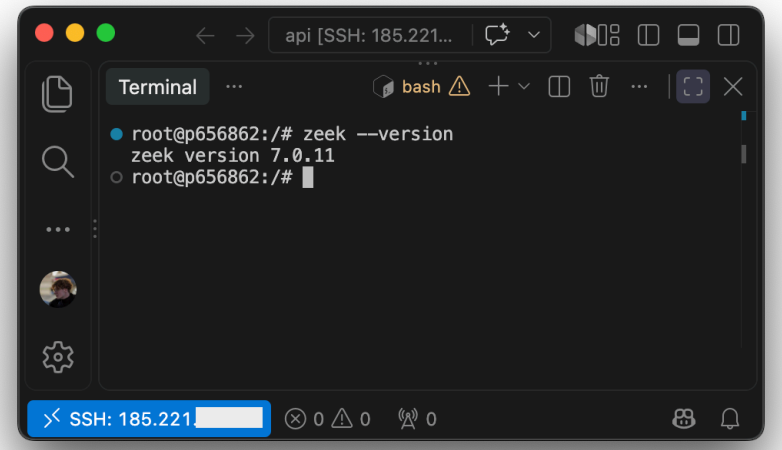


Рисунок 3.5: Проверка установки zeek

По-умолчанию анализатор просматривает сетевой трафик, проходящий через сетевой интерфейс eth0, однако клиенты подключаются к tun0, поэтому необходимо настроить zeek на прослушивание пакетов пользователей виртуальной частной сети OpenVPN. Для такой настройки необходимо

обратиться к файлу node.cfg, который является основным конфигурационным файлом, в нем необходимо указать нужный интерфейс (рис. 3.6).

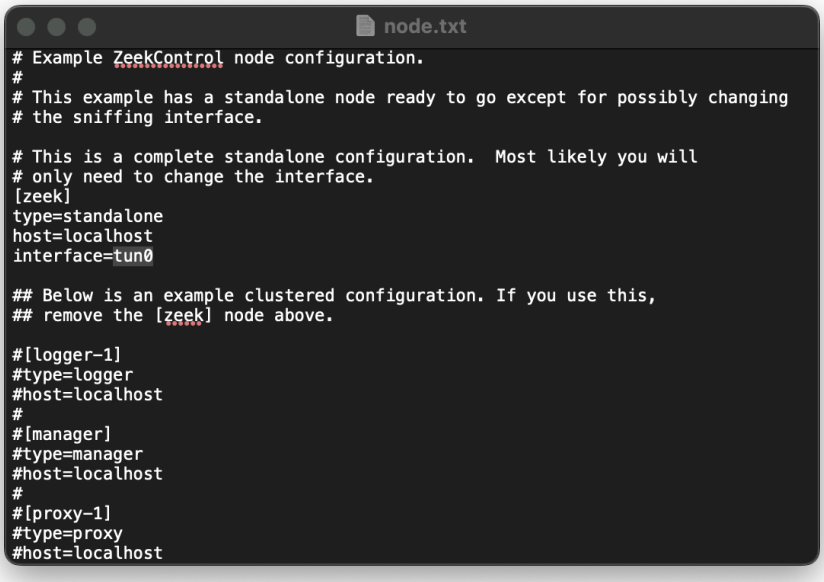


Рисунок 3.6: Изменение сетевого интерфейса

Также следует провести конфигурацию взаимодействия анализатора с базой данных SQLite. Так как инструмент Zeek позволяет создавать большое количество разнообразных журналов событий, то возникает проблема нагрузки базы данных информацией из неиспользуемых лог-файлов, создаваемых средством. Таким образом, был написан скрипт zeekDb, позволяющий сохранять данные из следующих источников: conn.log, dns.log, ssl.log, intel.log. На рисунке ниже показан созданный код (рис. 3.7).

Функция zeek_init выполняется один раз при старте Zeek. Внутри нее скрипт создает 4 фильтра, которые перехватывают системные логи и направляют их в базу данных. Для каждого фильтра заданы одинаковые параметры:

- $writer=Log::WRITER_SQLITE - указывает анализатору использовать модуль записи в SQLite;
- $path="путь к базе данных" - указывает путь к файлу базы данных.

Соответственно для каждого из вышеупомянутых лог-файлов создается отдельное правило, для их применения используются функции Log::add_filter.

Для подключения самописных скриптов и внешних плагинов в Zeek, необходимо

zeekDb.zeek

```
event zeek_init()
    {
    local filterConn: Log::Filter =
        [
        $name="sqlite",
        $path="/var/db/traffic",
        $config=table(["tablename"] = "conn"),
        $writer=Log::WRITER_SQLITE
        ];

    Log::add_filter(Conn::LOG, filterConn);

    local filterDns: Log::Filter =
        [
        $name="sqlite",
        $path="/var/db/traffic",
        $config=table(["tablename"] = "dns"),
        $writer=Log::WRITER_SQLITE
        ];

     Log::add_filter(DNS::LOG, filterDns);

     local filterSsl: Log::Filter =
        [
        $name="sqlite",
        $path="/var/db/traffic",
        $config=table(["tablename"] = "ssl"),
        $writer=Log::WRITER_SQLITE
        ];

     Log::add_filter(SSL::LOG, filterSsl);

     local filterIntel: Log::Filter =
        [
        $name="sqlite",
        $path="/var/db/traffic",
        $config=table(["tablename"] = "intel"),
        $writer=Log::WRITER_SQLITE
        ];

     Log::add_filter(Intel::LOG, filterIntel);
    }
```

Рисунок 3.7: Взаимодействие с базой данных

внести изменения в главный пользовательский конфигурационный файл local.zeek (рис. 3.8).

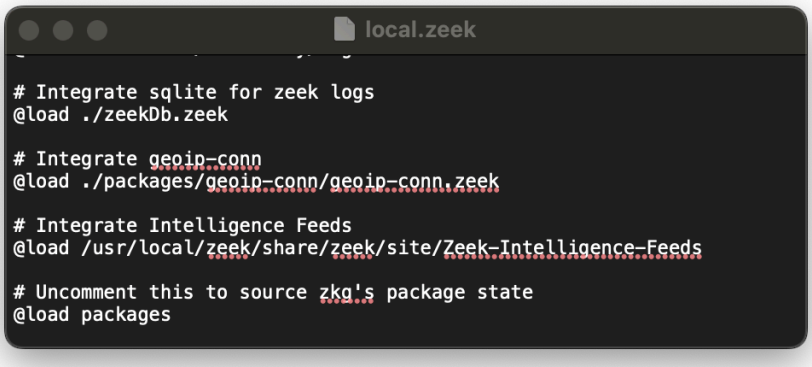


Рисунок 3.8: Файл local.zeek

Для установки пользовательского плагина geoip-conn используется стандартный менеджер пакетов Zeek. После успешной загрузки и сборки необходимо добавить данное расширение в файл local.zeek (рис. 3.8).

Также имеется расширение zeek-intelligence-feeds, в отличие от предыдущего, оно будет устанавливаться из репозитория системы контроля версий GitHub путем клонирования. После успешного переноса файлов плагина на сервер производится добавление данного подключаемого модуля в настроечный файл local.zeek (рис. 3.8). Стоит обратить внимание на содержимое склонированного репозитория, в нем содержатся перечни известных индикаторов компрометации, поскольку данные списки изменяются несколько раз в день, то создан скрипт zeek_update.sh (рис. 3.9).

Данный код принудительно обновляет базу индикаторов компрометации до самой актуальной версии из репозитория. Чтобы обеспечить выполнение скрипта 1 раз в 24 часа необходимо добавить задачу в планировщик cron (рис. 3.10).

В качестве проверки на успешную установку производится запуск zeek и проверка его статуса (рис. 3.11).

При успешной конфигурации плагинов в базе данных SQLite должны появится записи журналов событий, в conn информация о геолокации и в intel данные о подозрительных сетевых пакетах ([**bd-conn-intel-start?**]).

На рисунке выше видно, что при подключении клиента локальной частной сети с ip-адресом 10.8.0.8 в базе данных отображается информация из журналов событий, также в таблице conn добавлена информация о геолокации, что свидетельствует о работе geoip-conn. Помимо этого стоит обратить внимание на

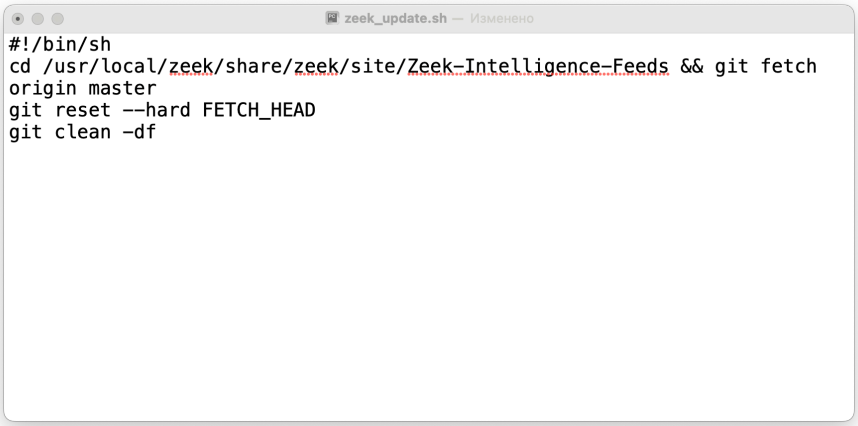


Рисунок 3.9: Содержимое файла zeek_update.sh

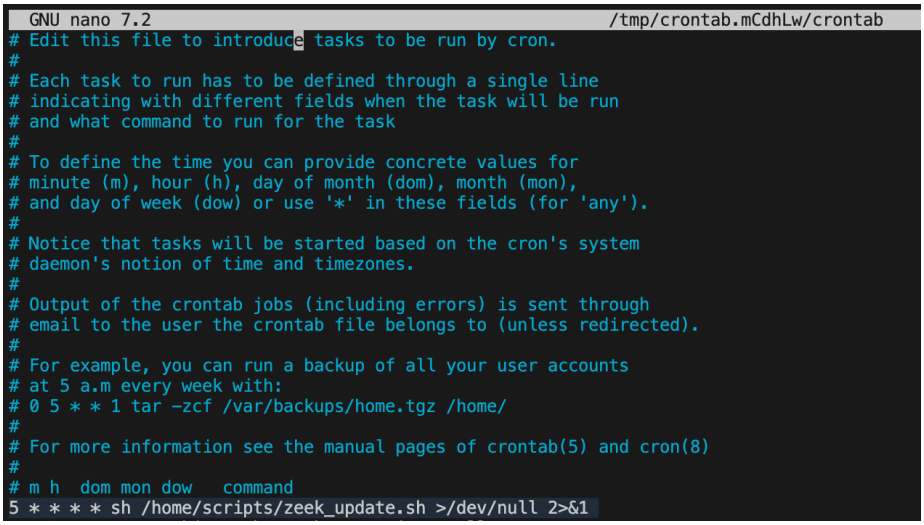


Рисунок 3.10: Добавление скрипта в планировщик

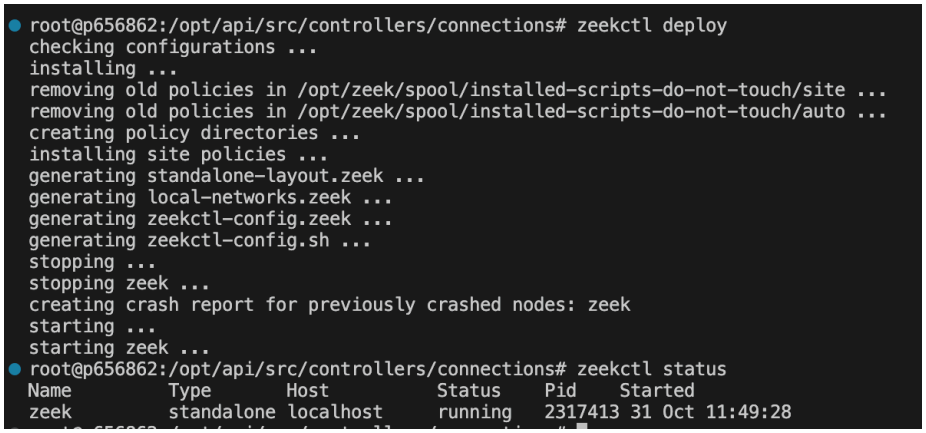


Рисунок 3.11: Запуск анализатора сетевого трафика zeek

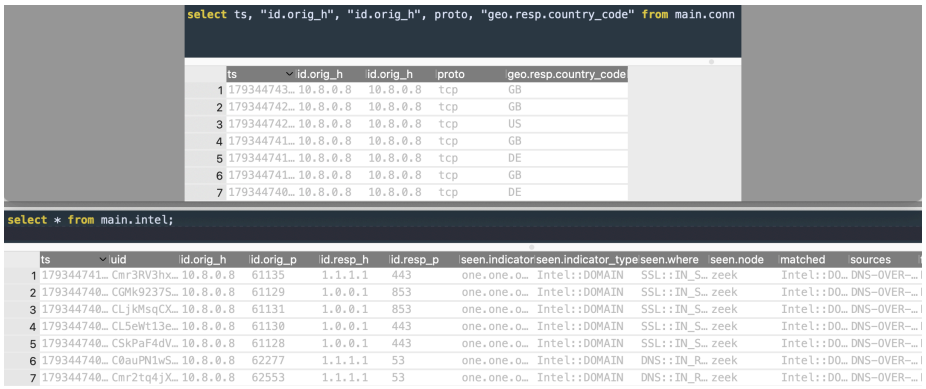


Рисунок 3.12: Журнал событий conn и intel. Проверка плагинов

intel, в котором имеются записи о подозрительных подключениях, данный факт говорит о работе расширения zeek-intelligence-feeds.

3.3 Конфигурация модуля хранения данных

В базе данных созданы 12 таблиц, ниже производится описание каждой из них.

Хранение базовой информации профиля пользователя вирутальной частной сети производится в vpn_users, данная таблица содержит следующие поля:

- id (первичный ключ, целочисленное значение) - индивидуальный идентификатор пользователя, присваивается клиенту в момент регистрации;
- ip_addr (первичный ключ, строковое значение) - ip-адрес, выданный пользователю регистрации;
- username (строковое значение) - придуманное клиентом имя, которое отображается в системе;
- hash_password (строковое значение) - захешированный пароль пользователя, благодаря которому клиент может входить в личный кабинет;
- role (строковое значение) - роль клиента в системе: администратор или узел;
- short_code (целочисленное значение) - короткий пин-код для быстрого входа в личный кабинет;

В разрабатываемой системе предусмотрен функционал добавления черных списков стран, ввиду этого создана таблица block_countries. Ниже представлены ее поля:

- id (первичный ключ, целочисленное значение) - индивидуальный идентификатор черного списка стран, выдается автоматически при создании;
- clientId (внешний ключ, целочисленные значение) - индивидуальный идентификатор пользователя виртуальной частной сети, связывает таблицу с vpn_users;
- countries_codes (строкое значение) - код страны.

Также стоит обратить внимание на таблицу block_domains, в которой описываются домены, добавленные в черный список, ниже представлены ее свойства:

- id (первичный ключ, целочисленное значение) - индивидуальный иденитификатор черного списка доменов, выдается автоматически при создании;
- clientId (внешний ключ, целочисленное значение) - индивидуальный идентификатор пользователя виртуальной частной сети, связывает таблицу с vpn_users;

- domains (строковое значение) - наименование домена.

В системе реализована возможность просмотра информации о сетевом трафике пользователя с ролью “узел” клиентом “администратор”, поэтому создана таблица node_depends, содержащая информацию о связях между членами сети. Ниже представлено описание свойств:

- id (первичный ключ, целочисленное значение) - индивидуальный идентификатор связи между пользователями, автоматически формируется при регистрации новой зависимости;
- node_id (внешний ключ, целочисленное значение) - индивидуальный идентификатор клиента с ролью “узел”;
- admin_id (внешний ключ, целочисленное значение) - индивидуальный идентификатор пользоваетеля с ролью “администратор”.

Для информирования и оповещения пользователя о событиях в системе, разработана таблица notifications, в которой содержатся уведомления для конкретного клиента. Ниже описаны свойства:

- id (первичный ключ, целочисленное значение) - индивидуальный идентификатор уведомления, формируется при регистрации нового сообщения;
- client_id (внешний ключ, целочисленное значение) - индивидуальный идентификатор клиента, которому адресовано конкретное извещение, необходим для связи с таблицой vpn_users;
- type (строковое значение) - тип уведомления;
- payload (строковое значение) - текст сообщения;
- is_read (целочисленное значение) - флаг прочтения уведомления, где 0 - не просмотрено, 1 - прочтено;

Хранение информации о метаданных зашифрованного соединения производится в таблице ssl, которая содержит следующие поля:

- ts (дробное значение) - временная метка (timestamp) события;
- uid (первичный ключ, строковое значение) - уникальный идентификатор соединения;
- id.orig_h (внешний ключ, строковое значение, не null) - IP-адрес инициатора (клиента) соединения, позволяет установить связь с таблицей vpn_users;
- id.orig_p (целочисленное значение) - порт инициатора соединения;
- id.resp_h (строковое значение) - IP-адрес отвечающего узла (сервера);
- id.resp_p (целочисленное значение) - порт отвечающего узла;
- version (строковое значение) - версия протокола;
- cipher (строковое значение) - используемый набор шифров;
- curve (строковое значение) - используемая эллиптическая кривая;
- server_name (строковое значение) - имя сервера;
- resumed (логическое значение) - флаг возобновления предыдущей сессии;

- last_alert (строковое значение) - последнее сообщение об ошибке в сессии;
- next_protocol (строковое значение) - согласованный протокол прикладного уровня;
- established (логическое значение) - флаг успешного установления SSL-соединения;
- ssl_history (строковое значение) - история состояний SSL-соединения;
- cert_chain_fps (строковое значение) - отпечатки цепочки сертификатов сервера;
- client_cert_chain_fps (строковое значение) - отпечатки цепочки сертификатов клиента;
- subject (строковое значение) - субъект сертификата сервера;
- issuer (строковое значение) - издатель сертификата сервера;
- client_subject (строковое значение) - субъект сертификата клиента;
- client_issuer (строковое значение) - издатель сертификата клиента;
- sni_matches_cert (логическое значение) - флаг соответствия имени сервера заявленному в сертификате;
- validation_status (строковое значение) - результат проверки валидности сертификата.

Информация из журнала событий conn.log анализатор сетевого трафика zeek хранится в соответствующей таблице conn, описание полей представлено ниже:

- ts (дробное значение) - временная метка начала соединения;
- uid (первичный ключ, строковое значение) - уникальный идентификатор сетевого соединения;
- id.orig_h (внешний ключ, строковое значение) - IP-адрес инициатора соединения, также необходим для связи с таблицей vpn_users;
- id.orig_p (целочисленное значение) - порт инициатора соединения;
- id.resp_h (строковое значение) - IP-адрес отвечающего узла;
- id.resp_p (целочисленное значение) - порт отвечающего узла;
- proto (строковое значение) - транспортный протокол;
- service (строковое значение) - идентифицированный протокол прикладного уровня;
- duration (дробное значение) - длительность соединения в секундах;
- orig_bytes (целочисленное значение) - количество байт полезной нагрузки, отправленных инициатором;
- resp_bytes (целочисленное значение) - количество байт полезной нагрузки, отправленных отвечающим узлом;
- conn_state (строковое значение) - статус состояния соединения;
- local_orig (логическое значение) - флаг принадлежности инициатора к локальной сети;
- local_resp (логическое значение) - флаг принадлежности отвечающего узла к локальной сети;
- missed_bytes (целочисленное значение) - количество потерянных в соединении байт;
- history (строковое значение) - история состояний соединения в виде символьных кодов;

- orig_pkts (целочисленное значение) - количество пакетов от инициатора;
- orig_ip_bytes (целочисленное значение) - общий объем IP-трафика (в байтах) от инициатора;
- resp_pkts (целочисленное значение) - количество пакетов от отвечающего узла;
- resp_ip_bytes (целочисленное значение) - общий объем IP-трафика (в байтах) от отвечающего узла;
- tunnel_parents (строковое значение) - идентификаторы туннелей, внутри которых инкапсулировано данное соединение;
- geo.orig.* и geo.resp.* (строковые, целочисленные и дробные значения) - набор полей геопозиционирования (страна, регион, город, координаты) и данных автономных систем (название организации) для инициатора и отвечающего узла соответственно.

Данные из лог-файла dns, генерируемый инструментом zeek, сохраняются в одноименной таблице базы данных:

- ts (дробное значение) - временная метка DNS-запроса;
- uid (первичный ключ, строковое значение) - уникальный идентификатор сетевого соединения;
- id.orig_h (внешний ключ, строковое значение) - IP-адрес клиента, используется для связи с таблицей vpn_users;
- id.orig_p (целочисленное значение) - порт клиента;
- id.resp_h (строковое значение) - IP-адрес DNS-сервера;
- id.resp_p (целочисленное значение) - порт DNS-сервера;
- proto (строковое значение) - транспортный протокол;
- trans_id (целочисленное значение) - идентификатор транзакции DNS;
- rtt (дробное значение) - время приема-передачи;
- query (строковое значение) - доменное имя, о котором запрашивается информация;
- qclass (целочисленное значение) - класс запроса;
- qclass_name (строковое значение) - строковое обозначение класса;
- qtype (целочисленное значение) - тип запроса;
- qtype_name (строковое значение) - строковое обозначение типа запроса;
- rcode (целочисленное значение) - код ответа сервера;
- rcode_name (строковое значение) - строковое обозначение кода ответа;
- AA (логическое значение) - флаг авторитетного ответа;
- TC (логическое значение) - флаг усеченного сообщения ;
- RD (логическое значение) - флаг запроса рекурсии;
- RA (логическое значение) - флаг доступности рекурсии;
- Z (целочисленное значение) - зарезервированное поле;
- answers (строковое значение) - полученные ответы на запрос;
- TTLs (строковое значение) - время жизни записей;
- rejected (логическое значение) - флаг отклонения DNS-запроса сервером.

Для хранения событий обнаружения индикаторов компрометации из лог-файла intel, сформирована одноименная таблица со следующими свойствами:

- ts (дробное значение) - временная метка обнаружения индикатора;
- uid (первичный ключ, строковое значение) - уникальный идентификатор соединения, в котором найдено совпадение;
- id.orig_h (внешний ключ, строковое значение) - исходный IP-адрес, создает связь с таблицей vpn_users;
- id.orig_p (целочисленное значение) - порт источника;
- id.resp_h (строковое значение) - целевой IP-адрес;
- id.resp_p (целочисленное значение) - целевой порт;
- seen.indicator (строковое значение) - зафиксированный индикатор компрометации;
- seen.indicator_type (строковое значение) - тип индикатора;
- seen.where (строковое значение) - контекст обнаружения;
- seen.node (строковое значение) - имя узла, зафиксировавшего совпадение;
- matched (строковое значение) - списки или категории, с которыми совпал индикатор;
- sources (строковое значение) - источники данных киберразведки;
- fuid (строковое значение) - уникальный идентификатор файла, если индикатор связан с файлом;
- file_mime_type (строковое значение) - MIME-тип обнаруженного файла;
- file_desc (строковое значение) - описание файла;
- checked (логическое значение) - флаг завершения проверки индикатора;
- cif.tags (строковое значение) - теги из системы Collective Intelligence Framework (далее CIF);
- cif.confidence (дробное значение) - уровень достоверности индикатора по данным CIF;
- cif.source (строковое значение) - источник данных CIF;
- cif.description (строковое значение) - описание угрозы из базы CIF;
- cif.firstseen (строковое значение) - дата первого обнаружения индикатора;
- cif.lastseen (строковое значение) - дата последнего обнаружения индикатора.

Модуль поведенческого профилирования при детектировании подозрительной активности клиента формирует событие, поэтому была сформирована таблица behavioral_alerts для хранения этих уведомлений. Ниже представлены ее поля:

- id (первичный ключ, целочисленное значение) - индивидуальный идентификатор уведомления;
- user_id (внешний ключ, целочисленное значение) - индивидуальный идентификатор клиента, необходим для связи с таблицей vpn_users;
- timestamp (значение типа timestamp) - дата и время создания сообщения;
- weekday (целочисленное значение) - день недели, представлен в виде значения от 0 до 6, где 0 - понедельник, 6 - воскресенье;
- slot (целочисленное значение) - часть дня, пиредставлен в виде значения от 0 до 3, где 0 - утро, 3 - ночь;
- anomaly_score (дробное значение) - показатель аномалии от 0 до 1, где 0

- сетевой трафик полностью соответствует поведению пользователя, 1 - имеются серьезные отклонения;

- details (строковое значение) - текст сообщения.

Для хранения информации о пользователе сети, которая в дальнейшем будет использована в обучении модели машинного обучения, создана таблица user_window_stats:

- id (первичный ключ, целочисленное значение) - индивидуальный идентификатор окна сбора информации;
- user_id (внешний ключ, целочисленное значение) - индивидуальный идентификатор клиента сети, необходим для связи с таблицей vpn_users;
- window_start (значение типа timestamp) - дата и время начала сбора данных о трафике;
- weekday (целочисленное значение) - день недели, представлен в виде значения от 0 до 6, где 0 - понедельник, 6 - воскресенье;
- slot (целочисленное значение) - часть дня, пиредставлен в виде значения от 0 до 3, где 0 - утро, 3 - ночь;
- total_conns (целочисленное значение) - общее число подключений;
- unique_dst_ips (целочисленное значение) - количество уникальных ip-адресов назначений;
- unique_dst_ports (целочисленное значение) - количество уникальных портов назначений;
- orig_bytes (целочисленное значение) - сумма полученных байт;
- resp_bytes (целочисленное значение) - сумма отправленных байт;
- avg_duration (дробное значение) - среднее время соединения;
- conns_per_sec (дробное значение) - количество соединений в секунду;
- created_at (значение типа timestamp) - дата и время создания записи;

В результате обучения модели машинного обучения создается файл с ее моделью, для сохранения информации по этому элементу создана таблица model_metadata:

- id (первичный ключ, целочисленное значение) - индивидуальный идентификатор версии модели;
- user_id (внешний ключ, целочисленное значение) - индивидуальный идентификатор клиента сети, необходим для связи с таблицей vpn_users;
- model_path (строковое значение) - путь в файловой системе сервера до обученной модели под конкретного пользователя;
- scaler_path (строковое значение) - путь к файлу скалера;

- training_date (timestamp значение) - дата последнего обучения модели машинного обучения клиента.

Для обеспечения корректного хранения данных, взаимодействие между таблицами реализовано следующим образом:

- Таблица vpn_users соединена с model_metadata связью типа «один-ко-многим» (внешний ключ user_id ссылается на первичный ключ id). Такая связь обусловлена сохранением истории: у одного клиента с течением времени может формироваться и сохраняться несколько версий обученных моделей машинного обучения;
- Таблица vpn_users соединена с user_window_stats связью типа «один-ко-многим» (внешний ключ user_id ссылается на первичный ключ id). Данный тип связи выбран, поскольку в процессе сбора статистики для одного пользователя генерируется множество срезов сетевого трафика;
- Таблица vpn_users соединена с behavioral_alerts связью типа «один-ко-многим» (внешний ключ user_id ссылается на первичный ключ id). Это необходимо из-за того, что модуль профилирования может фиксировать несколько событий с подозрительным поведением для одного клиента;
- Таблица vpn_users соединена с notifications связью типа «один-ко-многим» (внешний ключ client_id ссылается на первичный ключ id). Связь гарантирует, что одному пользователю может быть адресовано неограниченное количество системных уведомлений с возможностью индивидуального отслеживания статуса их прочтения;
- Таблица vpn_users соединена с node_depends связью типа «один-ко-многим», (внешние ключи node_id и admin_id ссылаются на первичный ключ id). Такая структура обусловлена тем, что один администратор может мониторить множество узлов, а один узел может контролироваться несколькими администраторами;
- Таблица vpn_users соединена с block_countries связью типа «один-ко-многим» (внешний ключ clientId ссылается на первичный ключ id). У одного пользователя может быть несколько стран в черном списке;
- Таблица vpn_users соединена с block_domains связью типа «один-ко-многим» (внешний ключ clientId ссылается на первичный ключ id). Аналогично предыдущей, эта связь позволяет задавать множество запрещенных доменных имен для одного конкретного клиента;
- Таблица vpn_users соединена с conn связью типа «один-ко-многим» (внешний ключ id.orig_h ссылается на первичный ключ ip_addr). Один клиент генерирует тысячи сетевых соединений;
- Таблица vpn_users соединена с dns связью типа «один-ко-многим» (внешний ключ id.orig_h ссылается на первичный ключ ip_addr). В рамках работы одного клиента отправляется множество DNS-запросов;

- Таблица vpn_users соединена с ssl связью типа «один-ко-многим» (внешний ключ id.orig_h ссылается на первичный ключ ip_addr). Один пользователь инициирует множество ssl-сессий;
- Таблица vpn_users соединена с intel связью типа «один-ко-многим» (внешний ключ id.orig_h ссылается на первичный ключ ip_addr). Эта связь обусловлена тем, что в трафике одного пользователя может быть зафиксировано множество сетевых соединений с обнаруженными индикаторами компрометации.

Исходя из описаний связей и таблиц базы данных, сформирована общая схема (рис. 3.13).

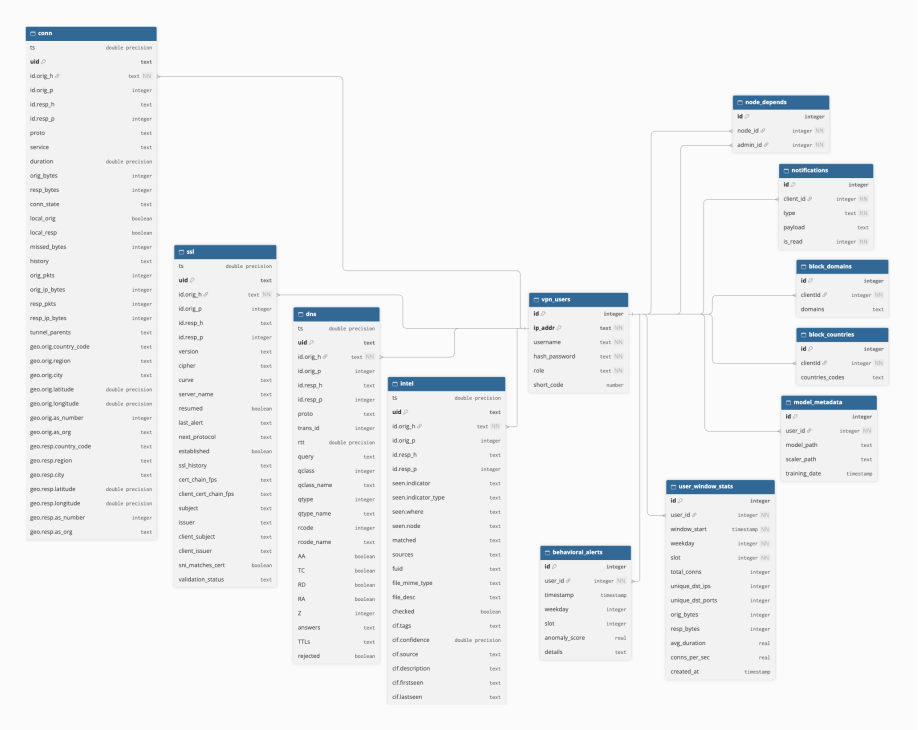


Рисунок 3.13: Схема базы данных

3.4 Создание модуля поведенческого профилирования

Модуль поведенческого профилирования состоит из пяти взаимосвязанных скриптов, которые в совокупности образуют полный цикл обработки данных, начиная от сбора статистики, заканчивая машинным обучением и выявлением аномалий в реальном времени. Ниже приведена таблица с описанием каждого файла (табл. 3.1).

Таблица 3.1: Описание скриптов из модуля поведенческого профилирования

Наименование	Назначение	Выполняемые функции	Используемые библиотеки
constants.py (см. Приложение B)	Хранение глобальных констант системы	Централизованное хранение пути к базе данных SQLite, позволяет легко изменять расположение источника без необходимости редактирования каждого скрипта по отдельности	Не использует сторонних библиотек
collector.py (см. Приложение C)	Сбор статистики, формирование исторических данных для обучения модели машинного обучения	Функция collector определяет текущее время и вычисляет временное окно за последние 6 часов. Запрашивает из базы данных список пользователей VPN, для которых еще не была обучена поведенческая модель. Для каждого такого пользователя агрегитрует сетевой трафик из таблицы conn и dns за последние 6 часов. Сохраняет агрегированную статистику в таблицу user_window_stats для последующего использования при обучении	sqlite3 - для подключения к базе данных и выполнения SQL-запросов time и datetime - для работы с временными метками, вычисления 6-часового интервала и извлечения текущего дня недели и часа.
helpers.py (см. Приложение D)	Вспомогательный модуль. Преобразует сырую статистику в нормализованный вектор признаков для алгоритмов машинного обучения	Функция get_user_history_rates извлекает исторические данные пользователя из таблицы user_window_stats. Переводит абсолютные значения в скорость (например, количество событий в секунду). Формирует циклические признаки. Возвращает готовый массив признаков	sqlite3 - для подключения к базе данных и выполнения SQL-запросов pandas - для загрузки результатов SQL-запроса в датафрейм numpy - для вычисления значений тригонометрических функций

Наименование	Назначение	Выполняемые функции	Используемые библиотеки
trainer.py (см. Приложение Е)	Создает индивидуальную модель машинного обучения для каждого пользователя	Функция user_train находит всех активных пользователей и запускает для них процес обучения Функция train_user_model запрашивает подготовленные исторические данные, используя модуль helpers.py Обучает модель изолирующий лес Сохраняет обученную модуль и скалер в файловую систему сервера Записывает путь к сохраненной модели в таблицу model_metadata	os - для создания директории сохранения моделей sqlite3 - для запросов к базе данных и сохроанения путей к моделям sklearn.ensemble.IsolationForest - основной алгоритм машинного обучения для поиска аномалий sklearn.preprocessing.StandardScaler - для стандартизации входных данных joblib - для сохранения файла обученной модели и скейлера

Наименование	Назначение	Выполняемые функции	Используемые библиотеки
checker (см. Приложение Б)	В режиме реального времени проверяет текущий трафик пользователей на предмет отклонений от их профиля	Функция user_check определяет список активных пользователей за последние 5 минут Функция checker собирает статистику сетевой активности конкретного пользователя за последние 5 минут за таблиц conn и dns Расчитывает текущие сетевые параметры трафика Загружает ранее обученную модель и скейлер из файла, путь к которому расположен в таблице model_metadata Масштабирует новые данные и передает их в модель для предсказания Модель машинного обучения сопоставляет текущий трафик с профилем пользователя, возвращая значение от 1 до -1, где 1 - норма, а -1 - абсолютное отклонение. Программный порог со значением <= -0.5 служит триггером для фиксации этого отрицательного ответа. При срабатывании условия система автоматически сохраняет предупреждение об инциденте в таблицу behavioral_alerts	sqlite3 - для запросов к базе данных pandas и numpy - для трансформации сырых признаков в массивы, пригодные для подачи на вход модели машинного обучения, а также для расчета циклических признаков joblib - для загрузки в память обученной модели json - для упаковки подробной информации об инциденте в формат JSON. datetime - для определения текущего времени, вычисления окна в 5 минут

3.5 Формирование REST API

Данный модуль разрабатываемой системы необходим для взаимодействия пользователя с удаленным сервером путем отправки запросов на строгоопределенные конечные точки (endpoint-ы), которые обеспечивают выполнение требуемых операций и предоставляют доступ к сведениям из базы данных.

3.5.1 Настройка Node.js

Установка среды разработки Node.js осуществляется через репозиторий операционной системы Linux Ubuntu. Каждому экземпляру Node принадлежит собственная версия npm - это стандратный менеджер пакетов, который входит в состав экосистемы Node.js, благодаря ему появляется возможность установки дополнительных модулей, необходимых для проекта. Для проверки корректности загрузки и настройки инструментов, в консоль выводятся их версии (рис. 3.14).

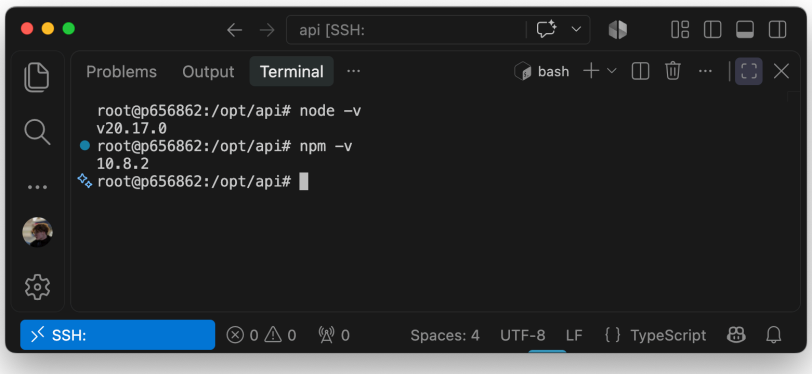


Рисунок 3.14: Версии Node.js и npm

3.5.2 Файловая организация подсистемы и программная реализация модуля

Файловая структура проекта представлена следующими директориями и конфигурационными файлами (рис. 3.15):

- node_modules - директория, содержащая все установленные соронние библиотеки и зависимости проекта, необходимые для корректного функционирования модуля;
- src - корневая папка исходного кода рарзрабатываемого серверного интерфейса. Директория разделена на подкаталоги:
  - controllers - модуль, содержащий контроллеры для обработки бизнес-логики входящих HTTP-запросов. Внутри реализовано логическое разделение на две подкатегории:
    - connections - содержит файлы, отвечающие за обработку данных, связанных с сетевой активностью пользователя;
    - user - содержит файлы, реализзующие логику ууправления учетными записями, процессами аутентификации, формированием

персональных черных списков и отправкой уведомлений;

  - db - директория для взаимодействия с харнилищем данных, содержит методы для формирования запросов к базе данных;
  - middleware - папка для промежуточных обработчиков, в данной реализации включает один файл, который реализует логику проверки токенов доступа и авторизации клиентских запрсоов перед их передачей в контроллеры;
  - types - содержит файлы с описанием структур данных;
  - index.ts - точка входа. В нем происходит инициализация, подключение обработчиков и регистрация маршрутов;
  - vpnConf.ts - специализированный скрипт, отвечающий за генерацию или выдачу клиентского VPN-профиля для подключения устройства;
- package.json/package-lock.json - файлы платформы Node.js, содержащие метаданные проекта, перечень установленных зависимостей и скрипты для сборки и запуска приложения;
- tsconfig.json - файл конфигурации компилятора TypeScript;
- nodemon.json - конфигурационный файл утилиты nodemon, которая используется на этапе разработки для автоматического перезапуска сервера при внесении изменений в исходный код.

Для понимания внутренних механизмов обработки HTTP-запросов и взаимодействия с хранилищем необходимо детально рассмотреть программную реализацию ключевых компонентов системы. Анализ кодовой базы целесообразно проводить в соответствии с потоком выполнения: от инициализации и фильтрации запросов до отработки бизнес-логики в контроллерах.

Файл src/index.ts (см. Приложение G) выступает в роли точки входа приложения и отвечает за инициализацию веб-сервера, подключение промежуточных обработчиков и регистрацию маршрутов REST API. Программная реализация выполнена на базе фреймворка Express.js. Скрипт создает экземпляр приложения Express и назначает сетевой порт 3000. Все входящие запросы до передачи контроллерам последовательно проходят через глобальные промежуточные обработчики:

- checkAuth (см. Приложение H) - функция, извлекает переданный клиентом ключ из заголовка Authorization и передает его вспомогательной функции mainAuth (см. Приложение I), которая сравнивает хеш-сумму полученного ключа с эталонным значением секретной фразы. При несовпадении хешей обработка прерывается, и клиенту возвращается статус 401 с ошибкой авторизации;
- bodyParser - выполняет парсинг тела HTTP-запроса, преобразуя переданные клиентом данные в объект JSON;

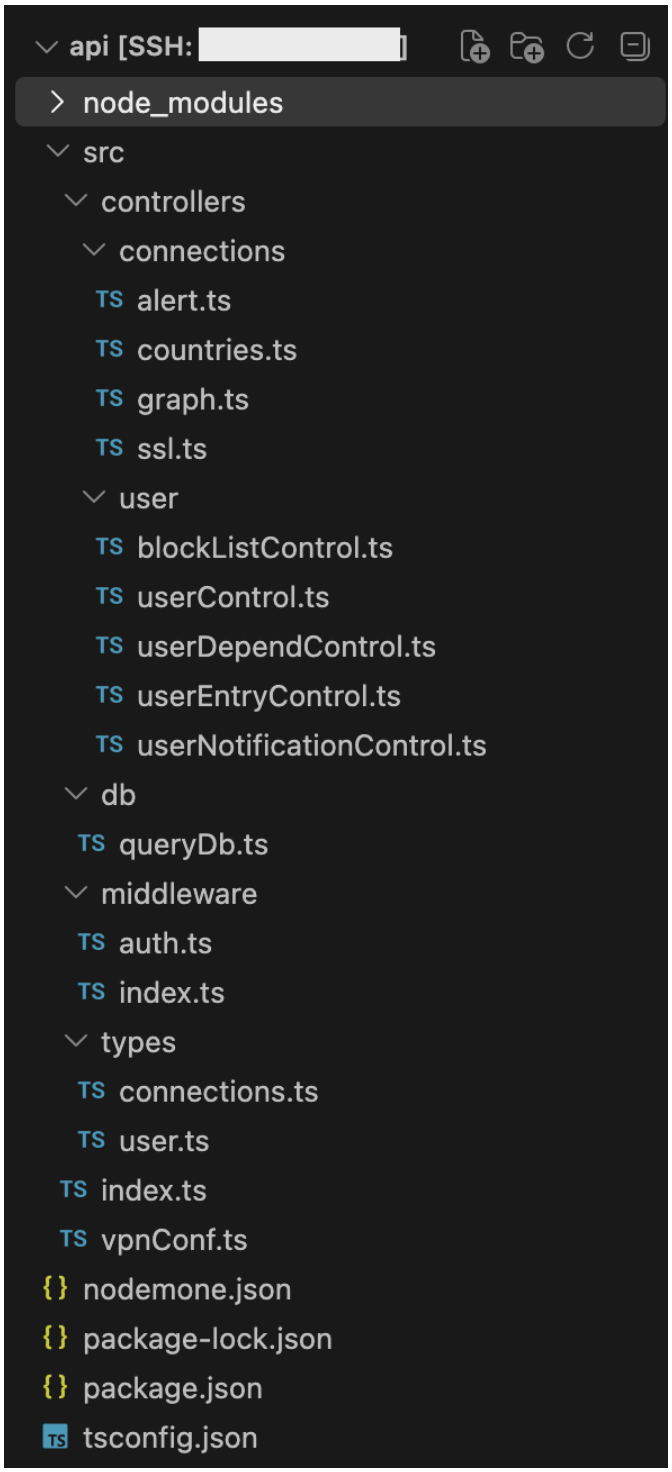


Рисунок 3.15: Файловая структура модуля

Маршрутизация построена на обработке POST-запросов и логически разделена на 2 фукциональные подсистемы:

- Сетевая аналитика и выявление угроз (/traffic/connections/):
  - graph - вызывает контроллер для получения агрегированной информации об объемах потребляемого трафика;
  - ssl/top и countries/top - возвращают рейтинги наиболее востребованных веб-ресурсов и стран взаимодействия;
  - alert - извлекает данные о выявленных проблемных соединениях, сетевых аномалиях и подозрительной активности пользователя.
- Управление учетными записями и доступом (/traffic/users/):
  - Маршруты reg, login, shortcode, fastEntry реализуют махенизмы регистрации, классического входа по паролю и быстрой авторизации в мобильном приложении по короткому коду;
  - info - предоставляет данные пользователя;
  - download/config - инициирует загрузку клиентского профиля OpenVPN на мобильное устройство;
  - blockList/countries, blockList/domains передают параметры для добавления и получениях пользовательских правил блокировки конкретных доменов и стран;
  - deleteDepends, getDepends, confirmDepend, denyDepend отвечают за установку, просмотр и разрыв связей между узлом и администратором;
  - sendNotiffication, getNotiffications обеспечивают процесс отправки и получения системных уведомлений.

Файл src/db/queryDb.ts (см. Приложение J) необходим для инкапсуляции логики подключения к базе данных и отправке в нее запросов.

Файл vpnConf.ts (см. Приложение K) необходим для автоматизации рутинного процесса создания конфигурационных файлов для безопаного подключения клиентов к серверу. При запросе скрипт обращается к утилите easy-rsa, которая генерирует сертификат. Затем программный код считывает общий базовый шаблон настроек OpenVPN-клиента, очищает его от лишних комментариев и добавляет сгенерированный ключ. Итоговый результат объединяется в единый готовый файл с расширением .ovpn и сохраняется в файловую систему сервера.

Файл src/controllers/user/userEntryControl.ts (см. Приложение L) обрабатывает первичную регистрацию новых пользователей, сохраняя их данные в базу, и при успешном создании аккаунта автоматически инициирует выпуск персонального VPN-профиля. Для уже зарегистрированных клиентов склипт реализует проверку связки логина и пароля, а также предоставляет функционал настройки и использования короткого цифрового кода. Это позволяет реализовать механизм

быстрого входа в мобильное приложение при последующих сессиях, избавляя пользователя от необходимости каждый раз вводить полный пароль.

Файл src/controllers/user/userDependControl.ts (см. Приложение M) управляет зависимостями между учетными записями в системе, а именно между управляемыми устройствами и их администраторами. Скрипт просматривает существующие соединения, а также обрабатывает логику их подтверждения, отклонения и удаления.

В файле src/controllers/user/userNotificationControl.ts (см. Приложение N) реализуется механизм отправки системных сообщений клиентам, а также предоставляет функционал для получения списка актуальных, не прочитанных уведомлений.

Файл src/controllers/user/userControl.ts (см. Приложение O) предоставляет клиенту функционал для скачивания VPN-конфигурации и получения базовой информации о пользователе.

Задача файла src/controllers/user/blockListControl.ts (см. Приложение P) заключается в управлении персональными черными списками для стран и доменных имен. Скрипт предоставляет функционал для сохранения, точечного обновления и получения списков ограничений, привязанных к конкретной учетной записи.

Файл src/controllers/connections/alert.ts (см. Приложение Q) отвечает за извлечение из базы данных и отправку клиенту информации о выявленных проблемных соединениях и сетевых аномалиях. Контроллер getAlerts обращается к таблице vpn_users для определения IP-ареса пользователя. Если адрес найден, он запрашивает из таблицы intel данные об угрозах, зафиксированных плагинов Zeek–Intelligence-Feeds. Дополнительно из таблицы behavioral_alerts извлекаются инциденты, зафиксированные модулем поведенческого профилирования на основе машинного обучения. Оба массива данных отправдяются в формате JSON.

Файл src/controllers/connections/countries.ts (см. Приложение R) предоставляет статистику по странам, с которыми взаимодействовал клиент, формируя рейтинг по количеству соединений с ip-адресами из конкретного региона. Функция getCountriesTop определяет роль и IP-адрес клиента. В базе данных выполняется запрос к таблице conn, подсчитывающий количество соединений для каждой страны за указанный период времени. Скрипт также запрашивает актуальный список стран, которые расположены в черном списке и помечает в итоговом ответе те регионы, которые в нем находятся.

Файл src/controllers/connections/graph.ts (см. Приложение S) реализует логику сбора данных об объемах входящего и исходящего сетевого трафика для их визуализации в виде графиков на стороне мобильного приложения. Скрипт выполняет SQL-запросы к таблице conn и возвращает объект, который содержит массив точек для графика.

Файл src/controllers/connections/ssl.ts (см. Приложение T) формирует рейтинг

наиболее посещаемых пользователем веб-ресурсов. Контроллер getSslTop запрашивает из таблицы ssl информацию об установленных соединениях и подсчитывает количество обращений к каждому домену. Скрипт учитывает черный список доменов. В результате пользователю возвращается массив из 5 самых популярных ресурсов по обращению.

Приложение А

Скрипт парсинга status-server.log

```
import sqlite3
import sys
import os
from datetime import datetime

DB_PATH = '/var/db/traffic.sqlite'
STATUS_LOG = '/run/openvpn-server/status-server.log'

def parse_status_file(filepath):
    users = {}
    try:
        with open(filepath, 'r') as f:
            for line in f:
                if line.startswith('CLIENT_LIST,'):
                    parts = line.strip().split(',')
                    if len(parts) >= 4:
                        common_name = parts[1]
                        virtual_addr = parts[3]
                        if virtual_addr and common_name and common_name != 'UNDEF':
                            users[common_name] = virtual_addr
    except Exception as e:
        print(f"                : {e}", file=sys.stderr)
        return None
    return users
```

```
def update_db(conn, current_users):
    cursor = conn.cursor()

    cursor.execute('SELECT username, ip_addr FROM vpn_users')
    db_records = {row[0]: row[1] for row in cursor.fetchall()}

    updated = 0

    for username, ip in current_users.items():
        if username in db_records:
            cursor.execute('''
                UPDATE vpn_users
                SET ip_addr = ?
                WHERE username = ?
            ''', (ip, username))
            updated += 1

    conn.commit()
    return updated


def main():
    db_dir = os.path.dirname(DB_PATH)
    if db_dir and not os.path.exists(db_dir):
        os.makedirs(db_dir, exist_ok=True)

    users = parse_status_file(STATUS_LOG)

    if users is None:
        print(f"{datetime.now().isoformat()}:                ,           ", file=sys.stder
        sys.exit(1)

    conn = sqlite3.connect(DB_PATH)

    updated = update_db(conn, users)

    print(f"{datetime.now().isoformat()}:      : {updated}")
    conn.close()


if __name__ == '__main__':
    main()
```

Приложение В

Содержимое constants.py

```
DB_PATH = '../traffic.sqlite'
```

Приложение С

Содержимое collector.py

```
import sqlite3
import time
from datetime import datetime
from constants import DB_PATH


def collector():
    ts_current = time.time()
    ts_back_6h = ts_current - 6 * 3600
    weekday = datetime.now().weekday()
    slot = datetime.now().hour // 6

    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute('''SELECT vpn_users.id, vpn_users.ip_addr, vpn_users.username FROM vpn_us
                    LEFT JOIN model_metadata ON vpn_users.id = model_metadata.user_id
                    WHERE vpn_users.ip_addr IS NOT NULL
                    AND model_metadata.model_path IS NULL''')

    users = cursor.fetchall()

    if (len(users) != 0):
        for user in users:
            cursor.execute('''SELECT
                            COUNT(*) AS total_conns,
                            SUM(orig_bytes),
                            SUM(resp_bytes),
                            duration,
```

```
                                COUNT(DISTINCT "id.resp_h") AS unique_dst_ips,
                                COUNT(DISTINCT "id.resp_p") AS unique_dst_ports,
                                AVG(duration) AS avg_duration,
                                (COUNT(*) / 6 * 60 * 60) AS conns_per_sec
                                FROM conn
                                WHERE conn."id.orig_h" = ? AND conn.ts <= ? AND conn.ts >= ?''',
                               (user[1], ts_current, ts_back_6h))

            userWindowData = cursor.fetchall()

            if (len(userWindowData[0]) != 0 and userWindowData[0][0] != 0):
                userWindowData = userWindowData[0]

                cursor.execute('''SELECT
                                COUNT(*) AS total_dns,
                                COUNT(DISTINCT query) AS unique_dns,
                                COALESCE(SUM(CASE WHEN rcode_name = 'NXDOMAIN' THEN 1 ELSE 0
                                FROM dns
                                WHERE "id.orig_h" = ? AND ts <= ? AND ts >= ?''',
                               (user[1], ts_current, ts_back_6h))

                dns_data = cursor.fetchone()
                total_dns = dns_data[0] if dns_data else 0
                unique_dns = dns_data[1] if dns_data else 0
                nxdomain_dns = dns_data[2] if dns_data else 0

                cursor.execute('''
                INSERT INTO user_window_stats
                (user_id, window_start, weekday, slot, total_conns, unique_dst_ips, unique_d
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                ''',
                               (user[0], ts_back_6h, weekday, slot, userWindowData[0], userW
                                userWindowData[5], userWindowData[2], userWindowData[1], use
                                userWindowData[7], total_dns, unique_dns, nxdomain_dns))
                conn.commit()

    cursor.close()


collector()
```

Приложение D

Содержимое helpers.py

```
import sqlite3
import pandas as pd
import numpy as np
from constants import DB_PATH


def get_user_history_rates(user_id, window_seconds=21600):
    query = """
        SELECT total_conns, unique_dst_ips, unique_dst_ports,
               orig_bytes, resp_bytes, avg_duration, conns_per_sec,
               total_dns, unique_dns, nxdomain_dns,
               weekday, slot, user_id
        FROM user_window_stats
        WHERE user_id = ?
        ORDER BY window_start
    """

    with sqlite3.connect(DB_PATH) as conn:
        df = pd.read_sql_query(query, conn, params=(user_id,))

    if len(df) < 10:
        return None

    df['conns_rate'] = df['total_conns'] / window_seconds
    df['ips_rate'] = df['unique_dst_ips'] / window_seconds
    df['ports_rate'] = df['unique_dst_ports'] / window_seconds
    df['orig_bytes_rate'] = df['orig_bytes'] / window_seconds
    df['resp_bytes_rate'] = df['resp_bytes'] / window_seconds
```

```
    df['dns_rate'] = df['total_dns'] / window_seconds
    df['unique_dns_rate'] = df['unique_dns'] / window_seconds
    df['nxdomain_rate'] = df['nxdomain_dns'] / window_seconds

    df['day_sin'] = np.sin(2 * np.pi * df['weekday'] / 7)
    df['day_cos'] = np.cos(2 * np.pi * df['weekday'] / 7)
    df['slot_sin'] = np.sin(2 * np.pi * df['slot'] / 4)
    df['slot_cos'] = np.cos(2 * np.pi * df['slot'] / 4)

    features = [
        'conns_rate', 'ips_rate', 'ports_rate',
        'orig_bytes_rate', 'resp_bytes_rate',
        'avg_duration', 'conns_per_sec',
        'dns_rate', 'unique_dns_rate', 'nxdomain_rate',
        'day_sin', 'day_cos', 'slot_sin', 'slot_cos'
    ]

    return df[features].values
```

Приложение Е

Содержимое trainer.py

```
import os
import sqlite3
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import joblib

from helpers import get_user_history_rates
from constants import DB_PATH

MODELS_DIR = 'models'

def train_user_model(user_id):
    os.makedirs(MODELS_DIR, exist_ok=True)

    X = get_user_history_rates(user_id, window_seconds=21600)

    if X is None:
        print(f"                        {user_id}")
        return

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    model = IsolationForest(
        n_estimators=100,
        contamination=0.05,
        random_state=42,
        bootstrap=True
    )
```

```
    model.fit(X_scaled)

    path = os.path.join(MODELS_DIR, f"{user_id}.joblib")
    joblib.dump((model, scaler), path)

    with sqlite3.connect(DB_PATH) as conn:
        cursor = conn.cursor()
        cursor.execute("""
            REPLACE INTO model_metadata
            (user_id, model_path, training_date)
            VALUES (?, ?, datetime('now'))
        """, (user_id, path))
        conn.commit()

def user_train():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute('''SELECT id FROM vpn_users WHERE vpn_users.ip_addr IS NOT NULL''')

    users = cursor.fetchall()

    if (len(users) != 0):
        for user in users:
            user_id = user[0]
            train_user_model(user_id)

    cursor.close()

user_train()
```

Приложение F

Содержимое checker.py

```
import sqlite3
import pandas as pd
import numpy as np
import joblib
import json
from datetime import datetime

from constants import DB_PATH


def checker(user_id, user_ip):
    with sqlite3.connect(DB_PATH) as conn:
        cursor = conn.cursor()

        cursor.execute("SELECT model_path FROM model_metadata WHERE user_id = ?", (user_id,)
        row = cursor.fetchone()
        if not row:
            return
        model_path = row[0]

        query = """
            SELECT
                COUNT(*) as total_conns,
                COUNT(DISTINCT "id.resp_h") as unique_dst_ips,
                COUNT(DISTINCT "id.resp_p") as unique_dst_ports,
                COALESCE(SUM(orig_bytes), 0) as orig_bytes,
                COALESCE(SUM(resp_bytes), 0) as resp_bytes,
                COALESCE(AVG(duration), 0) as avg_duration
            FROM conn
```

```
            WHERE "id.orig_h" = ?
            AND ts >= datetime('now', '-5 minutes');
        """

        cursor.execute(query, (user_ip,))
        row = cursor.fetchone()

        total_conns = row[0]

        if total_conns == 0:
            return

        unique_dst_ips = row[1]
        unique_dst_ports = row[2]
        orig_bytes = row[3]
        resp_bytes = row[4]
        avg_duration = row[5]

        dns_query = """
            SELECT
                COUNT(*) as total_dns,
                COUNT(DISTINCT query) as unique_dns,
                COALESCE(SUM(CASE WHEN rcode_name = 'NXDOMAIN' THEN 1 ELSE 0 END), 0) as nxd
            FROM dns
            WHERE "id.orig_h" = ?
            AND ts >= datetime('now', '-5 minutes');
        """
        cursor.execute(dns_query, (user_ip,))
        dns_row = cursor.fetchone()

        total_dns = dns_row[0] if dns_row else 0
        unique_dns = dns_row[1] if dns_row else 0
        nxdomain_dns = dns_row[2] if dns_row else 0

        window_seconds = 300
        conns_per_sec = total_conns / window_seconds

        now = datetime.now()
        weekday = now.weekday()
        slot = now.hour // 6

        conns_rate = total_conns / window_seconds
        ips_rate = unique_dst_ips / window_seconds
        ports_rate = unique_dst_ports / window_seconds
        orig_bytes_rate = orig_bytes / window_seconds
```

```
        resp_bytes_rate = resp_bytes / window_seconds

        dns_rate = total_dns / window_seconds
        unique_dns_rate = unique_dns / window_seconds
        nxdomain_rate = nxdomain_dns / window_seconds

        day_sin = np.sin(2 * np.pi * weekday / 7)
        day_cos = np.cos(2 * np.pi * weekday / 7)
        slot_sin = np.sin(2 * np.pi * slot / 4)
        slot_cos = np.cos(2 * np.pi * slot / 4)

        features = [
            conns_rate, ips_rate, ports_rate,
            orig_bytes_rate, resp_bytes_rate,
            avg_duration, conns_per_sec,
            dns_rate, unique_dns_rate, nxdomain_rate,
            day_sin, day_cos, slot_sin, slot_cos
        ]

        X_current = np.array([features])

        try:
            model, scaler = joblib.load(model_path)
        except Exception as e:
            print(f"          {model_path}: {e}")
            return

        X_scaled = scaler.transform(X_current)
        prediction = model.predict(X_scaled)

        if prediction[0] <= -0.5:
            anomaly_score = model.score_samples(X_scaled)[0]

            details = {
                "time": now.isoformat(),
                "total_conns_5m": total_conns,
                "orig_bytes_5m": orig_bytes,
                "resp_bytes_5m": resp_bytes,
                "conns_per_sec_rate": round(conns_rate, 4),
                "unique_dst_ips": unique_dst_ips,
                "total_dns_5m": total_dns,
                "nxdomain_dns_5m": nxdomain_dns
            }

            cursor.execute("""
```

```
                INSERT INTO behavioral_alerts (user_id, timestamp, weekday, slot, anomaly_sc
                VALUES (?, datetime('now'), ?, ?, ?, ?)
            """, (user_id, weekday, slot, float(anomaly_score), json.dumps(details)))

            conn.commit()


def user_check():
    with sqlite3.connect(DB_PATH) as conn:
        cursor = conn.cursor()
        query = """
            SELECT DISTINCT u.id, u.ip_addr
            FROM conn c
            JOIN vpn_users u ON c."id.orig_h" = u.ip_addr
            WHERE c.ts >= datetime('now', '-5 minutes')
        """
        cursor.execute(query)
        active_users = [row[0] for row in cursor.fetchall()]

    for user_id, user_ip in active_users:
        try:
            checker(user_id, user_ip)
        except Exception as e:
            print(f"                    {user_id}: {e}")


user_check()
```

Приложение G

Содержимое файла index.ts

```
import express from 'express'
import {checkAuth} from './middleware/index.ts'
import bodyParser from 'body-parser';
import getConnectionInfo from './controllers/connections/graph.ts';
import { getSslTop } from './controllers/connections/ssl.ts';
import { getCountriesTop } from './controllers/connections/countries.ts';
import { getAlerts } from './controllers/connections/alert.ts';
import { downloadUserVpnConfig, getUserInfo } from './controllers/user/userControl.ts';
import { getBlockListCountries, getBlockListDomains } from './controllers/user/blockListCont
import { userFastEntry, userLogin, userRegistration, userSetShortCode } from './controllers/
import { userConfirmDepend, userDeleteDepends, userDenyDepend, userGetDepends } from './cont
import { userGetNotifications, userSendNotification } from './controllers/user/userNotificat

export const app = express();
const PORT = 3000;

app.use(checkAuth)
app.use(bodyParser.json())

app.post('/traffic/connections/graph', getConnectionInfo);

app.post('/traffic/connections/ssl/top', getSslTop)

app.post('/traffic/connections/countries/top', getCountriesTop)

app.post('/traffic/connections/alert', getAlerts)

app.post('/traffic/users/info', getUserInfo)
```

```
app.post('/traffic/users/blockList/countries', getBlockListCountries)

app.post('/traffic/users/blockList/domains', getBlockListDomains)

app.post('/traffic/users/reg', userRegistration)

app.post('/traffic/users/login', userLogin)

app.post('/traffic/users/shortcode', userSetShortCode)

app.post('/traffic/users/download/config', downloadUserVpnConfig)

app.post('/traffic/users/deleteDepends', userDeleteDepends)

app.post('/traffic/users/getDepends', userGetDepends)

app.post('/traffic/users/confirmDepend', userConfirmDepend)

app.post('/traffic/users/denyDepend', userDenyDepend)

app.post('/traffic/users/sendNotiffication', userSendNotification)

app.post('/traffic/users/getNotiffications', userGetNotifications)

app.post('/traffic/users/fastEntry', userFastEntry)

app.listen(PORT, '0.0.0.0');
```

Приложение Н

Функция checkAuth

```
import { mainAuth } from "./auth"
import { Request, Response, NextFunction } from 'express';


export const checkAuth = (req: Request, res: Response, next: NextFunction) => {
    let authToken = req.headers.authorization
    if (!!authToken) {
        authToken = authToken.split(' ')[1]
        if (mainAuth(authToken)) {
            next()
        } else {
            res.status(401).json({error: '          '})
        }
    } else {
        res.status(401).json({error: '          '})
    }
}
```

Приложение I

Функция mainAuth

```
import crypto from 'crypto'

export const mainAuth = (token: string) => {
    if (token === crypto.createHash('sha256').update('bvcxertyuiolkmnbvedhklcvb').digest('he
        return true
    } else {
        return false
    }
}
```

Приложение J

Содержимое файла queryDb.ts

```
import sqlite3 from 'sqlite3';
import { Response } from 'express'

export const runQuery = (db: sqlite3.Database, sql: string, params: any[]): Promise<sqlite3.
            db.run(sql, params, function(err) {
                if (err) {
                    console.log(err)
                    reject('                ')
                }
                else resolve(this);
            });
        });

export const getQuery = <T>(db: sqlite3.Database, sql: string, params: any[]): Promise<T[]>
    db.all(sql, params, (err, rows) => {
        if (err) {
            reject('                ');
        }
        else resolve(rows as T[]);
    });
});

export const createDb = (res: Response) => {
    return new sqlite3.Database('db.sqlite', (err) => {
        if (err) return res.status(500).json({ error: '              ' });
    });
}
```

Приложение К

Содержимое файла vpnConf.ts

```
import { exec } from 'child_process';
import fs from 'fs/promises'
import path from 'path'
import { promisify } from 'util';

const execPromise = promisify(exec);

const EASY_RSA_DIR = 'easy-rsa';
const CLIENT_COMMON_FILE = 'client-common.txt';
const PKI_INLINE_PRIVATE_DIR = path.join(EASY_RSA_DIR, 'pki', 'inline', 'private');
const CONFIGS_DIR = 'vpn';

export async function generateClientCertificate(clientName: string) {
  const command = `cd ${EASY_RSA_DIR} && ./easyrsa --batch --days=3650 build-client-full "$
  try {
    const { stdout } = await execPromise(command);
    return stdout;

  } catch (error) {

    throw error;
  }
}

export async function buildOvpFile(clientName: string) {
  const commonContent = await fs.readFile(CLIENT_COMMON_FILE, 'utf8');
  const filteredCommon = commonContent.split('\n')
    .filter(line => !line.trim().startsWith('#'))
    .join('\n');
```

```
  const inlinePath = path.join(PKI_INLINE_PRIVATE_DIR, `${clientName}.inline`);
  let inlineContent = '';
  try {
    inlineContent = await fs.readFile(inlinePath, 'utf8');
  } catch (err) {
    throw new Error(`Cannot read inline file for ${clientName}: ${(err as Error).message}`);
  }
  const filteredInline = inlineContent.split('\n')
    .filter(line => !line.trim().startsWith('#'))
    .join('\n');

  const resultFile = `${filteredCommon}\n${filteredInline}`
  const filePath = path.join(CONFIGS_DIR, `${clientName}.ovpn`);
  await fs.writeFile(filePath, resultFile, 'utf8');

  return resultFile;
}
```

Приложение L

Содержимое файла userEntryControl.ts

```
import {Response, Request} from 'express'
import { createDb, getQuery, runQuery } from '../../db/queryDb';
import { buildOvpFile, generateClientCertificate } from '../../vpnConf';
import { UserRoleType } from '../../types/user';

type UserRegReqType = {
    username: string,
    hash_password: string,
    role: UserRoleType
}

type UserLoginReqType = {
    username: string,
    hash_password: string
}

type UserShortCodeReqType = {
    username: string,
    code: string
}

type UserFastEntryReqType = {
    username: string,
    passCode: string
}
```

```
export const userRegistration = async (req: Request, res: Response) => {
    const {username, hash_password, role}: UserRegReqType  = req.body;

    const db = createDb(res)

    try {

        await runQuery(
            db,
            `CREATE TABLE IF NOT EXISTS vpn_users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            hash_password TEXT NOT NULL,
            role TEXT NOT NULL,
            ip_addr TEXT
            );`,
            []
        )

        const result = await runQuery(
            db,
            `INSERT INTO vpn_users (username, hash_password, role)
            SELECT ?, ?, ?
            WHERE NOT EXISTS (SELECT 1 FROM vpn_users WHERE username = ?);`,
            [username, hash_password, role, username]
        )

        if (result.changes === 0 ) {
            db.close();
            res.json({success: false, text: "                        "});

        } else {
            await generateClientCertificate(username);
            const ovpnContent = await buildOvpFile(username);

            res.setHeader('Content-Type', 'application/x-openvpn-profile');
            res.setHeader('Content-Disposition', `attachment; filename="${username}.ovpn"`);

            db.close()

            res.send({success: true, text: ovpnContent});
        }

    } catch (e) {
        db.close()
```

```
        res.json({
            success: false,
            text: "                  "
        })
    }
}

export const userLogin = async (req: Request, res: Response) => {
    const {username, hash_password}: UserLoginReqType = req.body

    const db = createDb(res)

    try {
        const userHashInfoResp = await getQuery<{
            hash_password: string
        }>(
            db,
            `SELECT hash_password FROM vpn_users
            WHERE username = ?`,
            [username]
        )

        db.close()

        const userHash = userHashInfoResp[0]

        if (userHash.hash_password !== hash_password) {
            res.json({success: false, text: '               '})
        } else {
            res.json({success: true})
        }

    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: "                  "
        })
    }
}

export const userSetShortCode = async (req: Request, res: Response) => {
    const {username, code}: UserShortCodeReqType = req.body;
```

```
    const db = createDb(res)

    try {
        const idUserResp = await getQuery<{
            id: number
        }>(
            db,
            `SELECT id FROM vpn_users WHERE username = ?;`,
            [username]
        )

        const idUser = idUserResp[0].id

        if (!!idUser) {
            await runQuery(
                db,
                `UPDATE vpn_users SET short_code = ? WHERE username = ?;`,
                [code, idUser]
            )

            db.close()

            res.json({success: true, info: []})
        }

    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: "                "
        })
    }
}

export const userFastEntry = async (req: Request, res: Response) => {
    const {username, passCode}: UserFastEntryReqType = req.body

    const db = createDb(res)

    try {
        const dataResp = await getQuery<{
            short_code: string
        }>(
            db,
```

```
            `SELECT short_code FROM vpn_users WHERE username = ?`,
            [username]
        )

        const code = dataResp[0].short_code

        if (Number(code) !== Number(passCode)) {
            res.json({
                success: false,
                text: '       '
            })
        } else {
            res.json({
                success: true,
                info: []
            })
        }

    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: '                 '
        })
    }
}
```

Приложение М

Содежимое файла userDependControl.ts

```
import {Response, Request} from 'express'
import { createDb, getQuery, runQuery } from '../../db/queryDb'
import { NotificationType, UserRoleType } from '../../types/user'

type UserDeleteDependsReqType = {
    nodeId: number,
    adminId: number
}

type UserGetDependsReqType = {
    clientId: number,
    role: UserRoleType
}

type UserConfirmDependReqType = {
    nodeId: number,
    adminId: number,
    notificationId: number
}

type UserDenyDependReqType = {
    notificationId: number
}

export const userDeleteDepends = async (req: Request, res: Response) => {
    const {nodeId, adminId}: UserDeleteDependsReqType = req.body;
```

```
    const db = createDb(res)

    try {
        await runQuery(
            db,
            'DELETE FROM node_depends WHERE node_id = ? AND admin_id = ?',
            [nodeId, adminId]
        )

        db.close()

        res.json({
            success: true,
            info: []
        })
    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: '                  '
        })
    }
}

export const userGetDepends = async (req: Request, res: Response) => {
    const {clientId, role}: UserGetDependsReqType = req.body;

    const db = createDb(res)

    try {
        const dependIds = await getQuery<{
            id: number,
            username: string
        }>(
            db,
            `
            SELECT t1.id, t1.username
            FROM vpn_users AS t1
            INNER JOIN node_depends AS t2
            ON t1.id = t2.${role === 'node' ? 'admin_id' : 'node_id'}
            WHERE ${role === 'node' ? 'node_id' : 'admin_id'} = ?;
            `,
            [clientId]
        )
```

```
        db.close()

        res.json({
            success: true,
            info: dependIds
        })
    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: '                  '
        })
    }
}

export const userConfirmDepend = async (req: Request, res: Response) => {
    const {nodeId, adminId, notificationId}: UserConfirmDependReqType = req.body;

    const db = createDb(res)

    try {
        const notification = await getQuery<{
            id: number,
            client_id: number,
            type: NotificationType,
            payload: string,
            is_read: boolean
        }>(
            db,
            'SELECT * FROM notifications WHERE id = ?',
            [notificationId]
        )

        if (notification.length > 0) {
            await runQuery(
                db,
                'DELETE FROM notifications WHERE id = ?',
                [notificationId]
            )

            const depend = await getQuery<{
                id: number,
                node_id: number,
                admin_id: number
```

```
            }>(
                db,
                'SELECT * FROM node_depends WHERE node_id = ? AND admin_id = ?',
                [nodeId, adminId]
            )

            if (depend.length === 0) {
                await runQuery(
                    db,
                    'INSERT INTO node_depends (node_id, admin_id) VALUES (?, ?)',
                    [nodeId, adminId]
                )
            }
        }

        db.close()

        res.json({
            success: true,
            info: []
        })

    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: '                  '
        })
    }
}

export const userDenyDepend = async (req: Request, res: Response) => {
    const {notificationId}: UserDenyDependReqType = req.body;

    const db = createDb(res)

    try {
        const notificationArr = await getQuery<{
            id: number,
            client_id: number,
            type: NotificationType,
            payload: string,
            is_read: boolean
        }>(
```

```
            db,
            'SELECT * FROM notifications WHERE id = ?',
            [notificationId]
        )

        if (notificationArr.length > 0) {
            await runQuery(
                db,
                'DELETE FROM notifications WHERE id = ?',
                [notificationId]
            )
        }

        db.close()

        res.json({
            success: true,
            info: []
        })

    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: '                  '
        })
    }
}
```

Приложение N

Содержимое файла userNotificationControl.ts

```
import {Response, Request} from 'express'
import { createDb, getQuery, runQuery } from '../../db/queryDb';
import { NotificationType, UserRoleType } from '../../types/user';

type UserSendNotificationReqType = {
    clientId: number,
    type: NotificationType,
    payload: string
}

type UserGetNotificationsReqType = {
    clientId: number
}

export const userSendNotification = async (req: Request, res: Response) => {
    const {clientId, type, payload}: UserSendNotificationReqType = req.body

    const db = createDb(res)

    try {
        const clientExist = await getQuery<{
            id: number,
            username: string,
            hash_password: string,
            role: UserRoleType,
            ip_addr: string,
```

```
            short_code: string
        }>(
            db,
            `SELECT * FROM vpn_users WHERE id=?`,
            [clientId]
        )

        if (clientExist.length > 0) {
            const result = await runQuery(
                db,
                `INSERT OR IGNORE INTO notifications (client_id, type, payload) VALUES (?, ?
                [clientId, type, payload]
            )

            db.close()

            if (result.changes === 0) {
                res.json({
                    success: false,
                    text: '                '
                })
            } else {
                res.json({
                    success: true,
                    info: []
                })
            }
        } else {

            db.close()

            res.json({
                success: false,
                text: '                 '
            })
        }

    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: '                  '
        })
    }
```

```
}

export const userGetNotifications = async (req: Request, res: Response) => {
    const {clientId}: UserGetNotificationsReqType = req.body

    const db = createDb(res)

    try {
        const data = await getQuery<{
            clientId: number,
            id: number,
            type: NotificationType,
            payload: string,
            isRead: boolean
        }>(
            db,
            `SELECT client_id as clientId, id, type, payload, is_read as isRead FROM notific
            [clientId]
        )

        db.close()

        res.json({
            success: true,
            info: data
        })

    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: '                  '
        })
    }
}
```

Приложение О

Содержимое файла userControl.ts

```
import {Response, Request} from 'express'
import { createDb, getQuery } from '../../db/queryDb'
import { UserRoleType } from '../../types/user'
import * as fs from 'node:fs/promises'

type GetUserInfoReqType = {
    username: string
}

type UserDownloadConfReqType = {
    username: string
}

export const getUserInfo = async (req: Request, res: Response) => {
    const {username}: GetUserInfoReqType = req.body

    const db = createDb(res)

    try {
        const userInfo = await getQuery<{
            id: number,
            ipAddress: string,
            role: UserRoleType,
            username: string
        }>(
            db,
```

```
            `SELECT id, ip_addr AS ipAddress, role, username FROM vpn_users
            WHERE username = ?`,
            [username],
        )

        db.close();

        res.json({
            success: true,
            info: userInfo[0]
        });
    } catch (e) {
        db.close()

        res.json({
            success: false,
            text: '                  '
        })
    }
}

export const downloadUserVpnConfig = async (req: Request, res: Response) => {
    const {username}: UserDownloadConfReqType = req.body;

    try {
        const content = await fs.readFile(`/home/vpn/${username}.ovpn`, 'utf-8');

        res.json({
            success: true,
            info: content
        })
    } catch (e) {
        res.json({
            success: false,
            text: '                  '
        })
    }
}
```

Приложение Р

Содержимое файла blockListControl.ts

```
import { Response, Request } from 'express';
import { createDb, getQuery, runQuery } from '../../db/queryDb';

type BlockListCountriesType = {
    clientId: number,
    countryCodes: string,
    getAll: boolean
}

type BlockListDomainsType = {
    clientId: number,
    domains: {
        label: string,
        type: "edit" | "insert" | "delete",
        oldLabel: string
    },
    getAll: boolean
}

export const getBlockListCountries = async (req: Request, res: Response) => {
    const { clientId, countryCodes, getAll }: BlockListCountriesType = req.body;
    const db = createDb(res);

    try {
        await runQuery(
            db,
```

```
            `CREATE TABLE IF NOT EXISTS block_countries (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            clientId TEXT NOT NULL,
            countries_codes TEXT NOT NULL,
            UNIQUE(clientId, countries_codes)
            )`,
            []
        );

        if (!!getAll) {
            //
            const countries = await getQuery<{
                countries_codes: string
            }>(
                db,
                'SELECT countries_codes FROM block_countries WHERE clientId = ?',
                [clientId]
            );

            if (countries.length > 0) {
                res.json({ success: true, info: countries.map(c => c.countries_codes) });
            } else {
                res.json({ success: true, info: [] });
            }
        } else {
            //      /
            await runQuery(db, 'DELETE FROM block_countries WHERE clientId = ?', [clientId])

            if (countryCodes && countryCodes.trim().length > 0) {
                const codes = countryCodes.split(',');
                for (const code of codes) {
                    await runQuery(
                        db,
                        `INSERT INTO block_countries (clientId, countries_codes) VALUES (?,
                        [clientId, code.trim()]
                    );
                }
            }
            res.json({ success: true });
        }

        db.close();
    } catch (e) {
        db.close();
        res.json({
```

```
            success: false,
            info: '                  '
        });
    }
}

export const getBlockListDomains = async (req: Request, res: Response) => {
    const { clientId, domains, getAll }: BlockListDomainsType = req.body;
    const db = createDb(res);

    try {
        await runQuery(
            db,
            `CREATE TABLE IF NOT EXISTS block_domains (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            clientId TEXT NOT NULL,
            domains TEXT NOT NULL,
            UNIQUE(clientId, domains)
            )`,
            []
        );

        if (!!getAll) {
            //
            const getDomainsResp = await getQuery<{
                domains: string
            }>(
                db,
                'SELECT domains FROM block_domains WHERE clientId = ?',
                [clientId]
            );

            if (getDomainsResp.length > 0) {
                const getDomains = getDomainsResp.map((d, index) => {
                    return {
                        label: d.domains,
                        id: index
                    };
                });
                res.json({ success: true, info: getDomains });
            } else {
                res.json({ success: true, info: [] });
            }
        } else {
            //
```

```
            switch (domains.type) {
                case 'insert':
                    await runQuery(
                        db,
                        `INSERT OR IGNORE INTO block_domains (clientId, domains) VALUES (?,
                        [clientId, domains.label]
                    );
                    break;
                case 'delete':
                    await runQuery(
                        db,
                        `DELETE FROM block_domains WHERE clientId = ? AND domains = ?`,
                        [clientId, domains.label]
                    );
                    break;
                case 'edit':
                    await runQuery(
                        db,
                        `UPDATE block_domains SET domains = ? WHERE clientId = ? AND domains
                        [domains.label, clientId, domains.oldLabel]
                    );
                    break;
                default:
                    db.close();
                    return res.json({ success: false });
            }

            db.close();
            res.json({ success: true });
        }
    } catch (e) {
        db.close();
        res.json({
            success: false,
            info: '                  '
        });
    }
}
```

Приложение Q

Содержимое файла alert.ts

```
import {Response, Request} from 'express'
import { createDb, getQuery } from '../../db/queryDb';

type GetAlertsReqType = {
    timeStart: number,
    timeEnd: number,
    clientId: number
}

const pad = (num: number, size: number = 2) => String(num).padStart(size, '0');

function formatDateTime(ts: number) {
    const date = new Date(ts * 1000);

    const day = pad(date.getDate());
    const month = pad(date.getMonth() + 1);
    const year = date.getFullYear();
    const hours = pad(date.getHours());
    const minutes = pad(date.getMinutes());

    return `${day}.${month}.${year} ${hours}:${minutes}`;
}

const selectWeekText = (num: number): string => {
    switch (num) {
        case 0:
            return '      '
        case 1:
            return '    '
```

```
        case 2:
            return '   '
        case 3:
            return '    '
        case 4:
            return '    '
        case 5:
            return '    '
        case 6:
            return '      '
        default:
            return '            '
    }
}

const selectSlotText = (num: number): string => {
    switch (num) {
        case 0:
            return '  '
        case 1:
            return '  '
        case 2:
            return '   '
        case 3:
            return '  '
        default:
            return '            '
    }
}

export const getAlerts = async (req: Request, res: Response) => {

    const {timeStart, timeEnd, clientId}: GetAlertsReqType = req.body

    const db = createDb(res)

    try {
        const clientIpResp = await getQuery<{
            clientIp: string
        }>(
            db,
            'SELECT ip_addr as clientIp FROM vpn_users WHERE id = ?',
            [clientId]
        )
```

```
        const clientIp = clientIpResp[0]?.clientIp

        if (!!clientIp) {
            const alertsArr = await getQuery<{
                ts: number,
                ipSource: string,
                info: string,
                ioc: string
            }>(
                db,
                `SELECT ts, "id.resp_h" AS ipSource, sources AS info, "seen.indicator" AS io
                FROM intel
                WHERE "id.orig_h" = ? AND
                ts >= ? AND ts <= ?;`,
                [clientIp, timeStart, timeEnd]
            )

            const respAlert = alertsArr.map(alert => ({
                ...alert,
                labelTime: formatDateTime(alert.ts)
            }));

            const behavioralAlertsArr = await getQuery<{
                id: number,
                timestamp: number,
                weekday: number,
                slot: number,
                anomaly_score: number,
                details: string
            }>(
                db,
                `SELECT id, timestamp, weekday, slot, anomaly_score, details FROM behavioral
                WHERE user_id = ?;`,
                [clientId]
            )

            const respBehavioralAlertsArr = behavioralAlertsArr.map(alert => (
                {
                    ...alert,
                    weekday: selectWeekText(alert.weekday),
                    slot: selectSlotText(alert.slot)
                }
            ))

            db.close();
```

```
            res.json(
                {
                    success: true,
                    info: respAlert,
                    infoBehavioral: respBehavioralAlertsArr
                }
            )
        } else {
            db.close()

            res.json({
                success: true,
                info: []
            })
        }
    } catch (e) {
        db.close()

        res.json({
            success: false,
            info: '                  '
        })
    }
}
```

Приложение R

Содержимое файла countries.ts

```
import {Request, Response} from 'express'
import { createDb, getQuery } from '../../db/queryDb';
import { UserRoleType } from '../../types/user';

type CountriesTopReqType = {
    timeStart: number,
    timeEnd: number,
    limit: number,
    clientId: number
}

export const getCountriesTop = async (req: Request, res: Response) => {
    let adminId: null | number = null;
    const {timeStart, timeEnd, limit, clientId}: CountriesTopReqType = req.body

    const db = createDb(res)

    try {
        const clientIpResp = await getQuery<{
            clientIp: string,
            role: UserRoleType
        }>(
            db,
            'SELECT ip_addr as clientIp, role FROM vpn_users WHERE id = ?',
            [clientId]
        )

        const clientIp = clientIpResp[0]?.clientIp
        const clientRole = clientIpResp[0].role
```

```
        if (!!clientIp) {
            const countries = await getQuery<{
                country: string,
                count: number,
                lat: number,
                lon: number
            }>(
                db,
                `SELECT "geo.resp.country_code" AS country, COUNT(*) AS count, "geo.resp.lat
                FROM conn
                WHERE country IS NOT NULL AND
                ts >= ? AND ts <= ? AND
                "id.orig_h" = ?
                GROUP BY country
                ORDER BY count DESC;`,
                [timeStart, timeEnd, clientIp]
            )

            if (clientRole === 'node') {
                const clientDependResp = await getQuery<{
                    id: number
                }>(
                    db,
                    'SELECT admin_id as id FROM node_depends WHERE node_id = ?',
                    [clientId]
                )

                adminId = clientDependResp[0].id
            }

            let blackListCountriesResp = await getQuery<{
                countries: string
            }>(
                db,
                'SELECT countries_codes AS countries FROM block_countries WHERE clientId = 1
                [!!adminId ? adminId : clientId]
            )

            let blackListCountries = blackListCountriesResp[0].countries

            const blacklistSet = new Set(
                blackListCountries
                    ? blackListCountries.split(',').map(code => code.trim().toUpperCase()).f
                    : []
            );
```

```
            const enriched = countries.map(item => ({
                ...item,
                blackList: blacklistSet.has(item.country.toUpperCase())
            }));

            db.close()

            res.json({
                success: true,
                info: {
                    countries: enriched
                }
            })
        } else {
            db.close()

            res.json({
                success: true,
                info: {
                    countries: []
                }
            })
        }
    } catch (e) {
        db.close()

        res.json({
            success: false,
            info: '                  '
        })
    }
}
```

Приложение S

Содержимое файла graph.ts

```
import { createDb, getQuery } from "../../db/queryDb";
import {Response, Request} from 'express'
import { FilterViewType, RequestConnectionInfoParamsType } from "../../types/connections";

function formatLabelView(ts: number, view: FilterViewType) {
    const d = new Date(ts * 1000);
    switch (view) {
        case 'hour':
        case '6hour':
        case '12hour': {
            const h = d.getUTCHours().toString().padStart(2, '0');
            const m = d.getUTCMinutes().toString().padStart(2, '0');
            return `${h}:${m}`;
        }
        case 'day': {
            return `${d.getUTCHours().toString().padStart(2, '0')}:00`;
        }
        case 'week': {
            const days = [' ', ' ', ' ', ' ', ' ', ' ', ' '];
            return days[(d.getUTCDay() + 6) % 7];
        }
        case 'month': {
            return d.getUTCDate().toString().padStart(2, '0');
        }
        default: return '';
    }
}

function formatLabelCustom(ts: number, bucketSize: number) {
```

```
    const d = new Date(ts * 1000);
    if (bucketSize < 3600) {
        const h = d.getUTCHours().toString().padStart(2, '0');
        const m = d.getUTCMinutes().toString().padStart(2, '0');
        return `${h}:${m}`;
    } else if (bucketSize === 3600) {
        return `${d.getUTCHours().toString().padStart(2, '0')}:00`;
    } else {
        return d.toISOString().slice(0, 10); // YYYY-MM-DD
    }
}

function generateTitle(view: FilterViewType, startUnix: number, endUnix: number) {
    const start = new Date(startUnix * 1000);
    const end = new Date(endUnix * 1000);

    const pad = (n: number) => n.toString().padStart(2, '0');
    const fmtTime = (d: Date) => `${pad(d.getUTCHours())}:${pad(d.getUTCMinutes())}`;
    const fmtDate = (d: Date) => `${pad(d.getUTCDate())}.${pad(d.getUTCMonth() + 1)}.${d.get

    const monthNames = [
        '   ', '    ', '  ', '   ', '  ', '  ',
        '  ', '   ', '    ', '    ', '   ', '    '
    ];

    switch (view) {
        case 'hour':
            return ` ${fmtTime(start)}   ${fmtTime(end)} ${fmtDate(start)}`;
        case '6hour':
            return ` ${fmtTime(start)}   ${fmtTime(end)} ${fmtDate(start)}`;
        case '12hour':
            return ` ${fmtTime(start)}   ${fmtTime(end)} ${fmtDate(start)}`;
        case 'day':
            return fmtDate(start);
        case 'week': {
            const endOfWeek = new Date(endUnix * 1000 - 1000);
            return `    ${fmtDate(start)} - ${fmtDate(endOfWeek)}`;
        }
        case 'month': {
            const year = start.getUTCFullYear();
            const month = monthNames[start.getUTCMonth()];
            return `${month} ${year}`;
        }
        case 'custom': {
            const diffSec = endUnix - startUnix;
```

```
            if (diffSec <= 86400 && start.getUTCDate() === end.getUTCDate()) {
                return `  ${fmtTime(start)}   ${fmtTime(end)} ${fmtDate(start)}`;
            } else {
                return `  ${fmtDate(start)} ${fmtTime(start)}   ${fmtDate(end)} ${fmtTime(en
            }
        }
        default:
            return '';
    }
}

const aggregate = (
    rows: any[],
    bucketList: {
        ts: number;
        label: string;
        value: number;
    }[],
    bucketSize: number
) => {
    const result = bucketList.map(b => ({ ...b }));
    const map = new Map(result.map(b => [b.ts, b]));
    for (const row of rows) {
        const key = Math.floor(row.ts / bucketSize) * bucketSize;
        const entry = map.get(key);
        if (entry) {
            entry.value += row.bytes;
        }
    }
    return result;
};

const getConnectionInfo = async (req: Request, res: Response) => {
    const {
        clientId,
        selectedView,
        timeStart,
        timeEnd,
        offset,
        dateNow
    }: RequestConnectionInfoParamsType = req.body;

    const db = createDb(res)

    try {
```

```
        const ipResponse = await getQuery<{ipAddr: string}>(
            db,
            'SELECT ip_addr as ipAddr FROM vpn_users WHERE id = ?',
            [clientId]
        )

        const clientIp: undefined | string = ipResponse[0]?.ipAddr
        const nowSec = new Date(dateNow)
        const off = Number(offset) || 0;

        let intervalStart, intervalEnd, bucketSize, buckets;

        if (selectedView === 'custom') {
            intervalStart = timeStart;
            intervalEnd = timeEnd;
            const duration = intervalEnd - intervalStart;
            if (duration <= 3600) bucketSize = 600;
            else if (duration <= 21600) bucketSize = 900;
            else if (duration <= 43200) bucketSize = 1800;
            else if (duration <= 86400) bucketSize = 3600;
            else if (duration <= 7 * 86400) bucketSize = 86400;
            else bucketSize = 86400;

            const start = Math.floor(intervalStart / bucketSize) * bucketSize;
            buckets = [];
            for (let t = start; t < intervalEnd; t += bucketSize) {
                buckets.push({ ts: t, label: formatLabelCustom(t, bucketSize), value: 0 });
            }
        } else {
            const t = nowSec;
            switch (selectedView) {
                case 'hour':
                    intervalStart = Math.floor(t.getTime() / 3600) * 3600 - off * 3600;
                    intervalEnd = intervalStart + 3600;
                    bucketSize = 600;
                    break;
                case '6hour':
                    intervalStart = Math.floor(t.getTime() / 21600) * 21600 - off * 21600;
                    intervalEnd = intervalStart + 21600;
                    bucketSize = 900;
                    break;
                case '12hour':
                    intervalStart = Math.floor(t.getTime() / 43200) * 43200 - off * 43200;
                    intervalEnd = intervalStart + 43200;
                    bucketSize = 1800;
```

```
                    break;
                case 'day':
                    intervalStart = Math.floor(t.getTime() / 86400) * 86400 - off * 86400;
                    intervalEnd = intervalStart + 86400;
                    bucketSize = 3600;
                    break;
                case 'week': {
                    const d = new Date(t.getTime() * 1000);
                    const dayOfWeek = d.getUTCDay();
                    const diffToMonday = (dayOfWeek + 6) % 7;
                    const monday = new Date(Date.UTC(d.getUTCFullYear(), d.getUTCMonth(), d
                    const weekStart = monday.getTime() / 1000;
                    intervalStart = weekStart - off * 7 * 86400;
                    intervalEnd = intervalStart + 7 * 86400;
                    bucketSize = 86400;
                    break;
                }
                case 'month': {
                    const d = new Date(t.getTime() * 1000);
                    const year = d.getUTCFullYear();
                    const month = d.getUTCMonth();
                    const totalMonths = year * 12 + month - off;
                    const targetYear = Math.floor(totalMonths / 12);
                    const targetMonth = totalMonths % 12;
                    const startDate = new Date(Date.UTC(targetYear, targetMonth, 1));
                    intervalStart = startDate.getTime() / 1000;
                    const endDate = new Date(Date.UTC(targetYear, targetMonth + 1, 1));
                    intervalEnd = endDate.getTime() / 1000;
                    bucketSize = 86400;
                    break;
                }
                default:
                    return res.status(400).json({ error: `          selectedView: ${selected
            }

            buckets = [];
            for (let t = intervalStart; t < intervalEnd; t += bucketSize) {
                buckets.push({ ts: t, label: formatLabelView(t, selectedView), value: 0 });
            }
        }

        const title = generateTitle(selectedView, intervalStart, intervalEnd);

        const [origRows, respRows] = await Promise.all([
            getQuery(
```

```
            db,
            `SELECT ts, orig_bytes AS bytes FROM conn WHERE "id.orig_h" = ? AND ts BETWE
            UNION ALL
            SELECT ts, resp_bytes AS bytes FROM conn WHERE "id.resp_h" = ? AND ts BETWEE
            [clientIp, intervalStart, intervalEnd, clientIp, intervalStart, intervalEnd]
        ),
        getQuery(
            db,
            `SELECT ts, resp_bytes AS bytes FROM conn WHERE "id.orig_h" = ? AND ts BETWE
            UNION ALL
            SELECT ts, orig_bytes AS bytes FROM conn WHERE "id.resp_h" = ? AND ts BETWEE
            [clientIp, intervalStart, intervalEnd, clientIp, intervalStart, intervalEnd]
        ),
        getQuery(
            db,
            `SELECT MAX(bytes) AS max_bytes
            FROM (
                SELECT resp_bytes AS bytes
                FROM conn
                WHERE "id.orig_h" = ? AND ts BETWEEN ? AND ?

                UNION ALL

                SELECT orig_bytes AS bytes
                FROM conn
                WHERE "id.resp_h" = ? AND ts BETWEEN ? AND ?
            ) AS traffic_data;`,
            [clientIp, intervalStart, intervalEnd, clientIp, intervalStart, intervalEnd]
        )
    ]);

    const respPoints = aggregate(respRows, buckets, bucketSize)
    const origPoints = aggregate(origRows, buckets, bucketSize)

    const sumRespBytes = respPoints.reduce((acc, cur) => {
        return cur.value + acc
    }, 0)
    const sumOrigBytes = origPoints.reduce((acc, cur) => {
        return cur.value + acc
    }, 0)

    return res.json({
        success: true,
        title: title,
```

```
            info: {
                resp: {
                    points: respPoints,
                    max: Math.max(...respPoints.map(item => item.value)),
                    min: Math.min(...respPoints.map(item => item.value)),
                    sum: sumRespBytes
                },
                orig: {
                    points: origPoints,
                    max: Math.max(...origPoints.map(item => item.value)),
                    min: Math.min(...origPoints.map(item => item.value)),
                    sum: sumOrigBytes
                }
            }
        });
    } catch (e) {
        return res.json({ success: false, info: e });
    } finally {
        db.close();
    }
}

export default getConnectionInfo
```

Приложение Т

Содержимое файла ssl.ts

```
import {Response, Request} from 'express'
import { createDb, getQuery } from '../../db/queryDb'
import { UserRoleType } from '../../types/user';

type GetSslTopParamsType = {
    timeStart: number,
    timeEnd: number,
    limit: number,
    clientId: number
}

export const getSslTop = async (req: Request, res: Response) => {
    let adminId: null | number = null;
    const {
        timeStart,
        timeEnd,
        limit,
        clientId
    }: GetSslTopParamsType = req.body

    const db = createDb(res)

    try {
        const clientInfoResp = await getQuery<{
            clientIp: string,
            role: UserRoleType
        }>(
            db,
            'SELECT ip_addr as clientIp, role FROM vpn_users WHERE id = ?',
```

```
            [clientId]
        )

        const clientIp: undefined | string = clientInfoResp[0]?.clientIp
        const clientRole: UserRoleType = clientInfoResp[0].role

        if (!!clientIp) {
            const allSites = await getQuery<{
                server: string,
                value: number,
            }>(
                db,
                `SELECT server_name AS server, COUNT(*) AS value
                FROM ssl
                WHERE server IS NOT NULL AND
                "id.orig_h" = ? AND
                ts >= ? AND ts <= ?
                GROUP BY server;`,
                [clientIp, timeStart, timeEnd]
            )

            if (clientRole === 'node') {
                const clientDependResp = await getQuery<{
                    id: number
                }>(
                    db,
                    'SELECT admin_id as id FROM node_depends WHERE node_id = ?',
                    [clientId]
                )

                adminId = clientDependResp[0].id
            }

            const blacklistStrResp = await getQuery<{
                domains: string
            }>(
                db,
                'SELECT domains FROM block_domains WHERE clientId = ?',
                [!!adminId ? adminId : clientId]
            )

            let blacklistStr: string = blacklistStrResp[0]?.domains

            const blockedDomains = blacklistStr
                    ? blacklistStr.split(',').map(d => d.trim().toLowerCase()).filter(Boolea
```

```
                    : [];

            const enriched = allSites.map(site => {
                    const serverLower = site.server.toLowerCase();
                    const inBlacklist = blockedDomains.some(domain => serverLower.includes(d
                    return { ...site, blackList: inBlacklist };
                });

            enriched.sort((a, b) => b.value - a.value);

            const top5 = enriched.slice(0, 5);

            const blockedNotInTop5 = enriched.filter(
                site => site.blackList && !top5.some(t => t.server === site.server)
            );

            const result = [...top5, ...blockedNotInTop5];

            db.close()

            res.json({
                success: true,
                info: result
            })
        } else {
            db.close()

            res.json({
                success: true,
                info: []
            })
        }
    } catch (e) {
        db.close()

        res.json({
            success: false,
            info: '                '
        })
    }
}
```

1. Статистика утечек данных за 2025 год [Электронный ресурс]. F6. URL: https://www.f6.ru/media-center/press-releases/cyberthreats-2025/ (дата обращения: 10.10.2026).

2. Рост использования инструментов для организации защищенного доступа [Электронный ресурс]. Коммерсантъ. URL: https://www.kommersant.ru/doc/8624480 (дата обращения: 10.10.2026).
3. Актуальные киберугрозы: Россия [Электронный ресурс]. Positive Technologies. URL: https://ptsecurity.com/research/analytics/russia-cyberthreat-landscape-2026/ (дата обращения: 10.10.2026).
4. Применение методов машинного обучения для выявления аномалий в сетевом трафике [Электронный ресурс]. Cyberleninka. URL: https://cyberleninka.ru/article/n/primenenie-metodov-mashinnogo-obucheniya-dlya-vyyavleniya-anomaliy-v-setevom-trafike/viewer (дата обращения: 10.10.2026).
5. Машинное обучение с учителем и без: как выбрать правильный подход для вашего проекта [Электронный ресурс]. Сбербанк. URL: https://sberbs.ru/blogs/blog/mashinnoe-obuchenie-s-uchitelem-i-bez (дата обращения: 10.10.2026).
6. О Zeek [Электронный ресурс]. Zeek. URL: https://zeek.org/about/ (дата обращения: 10.10.2026).
7. Improving Time Series Prediction Models Using Cyclic Features Encoding in Neural Networks [Электронный ресурс]. Medium. URL: https://medium.com/@dhanyahari07/improving-time-series-prediction-models-using-cyclic-features-encoding-in-neural-networks-0eebef307fc2 (дата обращения: 10.10.2026).
8. Доля рынка трафика с мобильных устройств в сравнении с настольными компьютерами и планшетами [Электронный ресурс]. Similarweb. URL: https://www.similarweb.com/ru/platforms/ (дата обращения: 10.10.2026).